



Portal Administrador

Backend de gestión de inventario con arquitectura hexagonal, seguridad JWT y trazabilidad de calidad

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Departamento de Ciencias de la Computación
Aseguramiento de la Calidad de Software

Informe Técnico Don Gato Inventory

Commit documentado: 691fa0d

feat: initialize inventory management backend project
with hexagonal architecture and security configuration

66 archivos

17 endpoints

JWT + SQA

Equipo de trabajo

Mesias Mariscal
Gabriel Murillo
Denise Rea
Julio Viche

Mayo 2026

Índice general

1. Resumen Ejecutivo	1
1.1. Datos del Commit	1
1.2. Objetivo Funcional	1
2. Arquitectura Hexagonal	3
2.1. Idea Central	3
2.2. Capas y Responsabilidades	3
2.3. Flujo de Dependencias	4
3. Complejidad Técnica y Decisiones	5
3.1. Dominio Rico	5
3.2. Inventario Auditable	5
3.3. Complejidad Operacional	5
3.4. Transacciones	6
3.5. Modelo de Calidad de McCall	6
4. API REST y Endpoints	8
4.1. Reglas Generales	8
4.2. Tabla Completa de Endpoints	8
4.3. Contratos de Entrada	9
4.4. Enums Expuestos	10
4.5. Ejemplos de Uso	10
5. Seguridad	11
5.1. Autenticación JWT	11
5.2. Usuarios de Prueba	11
5.3. CORS y Sesiones	11
6. Persistencia y Datos	12
6.1. Modelo Persistente	12
6.2. Repositorios	12
6.3. Docker Compose	12

7. Manejo de Errores	13
7.1. Formato Común	13
7.2. Mapa de Excepciones	13
8. Pruebas, SQA y CI/CD	14
8.1. Pruebas Implementadas	14
8.2. Motor de Calidad McCall	14
8.3. Pipeline	14
9. Inventario de Archivos del Commit	16
9.1. Resumen por Tipo de Cambio	16
9.2. Lista de Archivos	16
A. Código Completo de los Archivos Activos	19
A.1. Configuración de Proyecto	19
A.2. Aplicación	24
A.3. Dominio	25
A.4. Aplicación: Puertos y Casos de Uso	29
A.5. Infraestructura: Persistencia	34
A.6. Infraestructura: Seguridad y Configuración	43
A.7. Interfaces REST: Controladores	48
A.8. Interfaces REST: DTOs, Errores y Mappers	52
A.9. SQA	60
A.10. Pruebas	62
B. Snapshots de Archivos Eliminados	72
B.1. Snapshot de aplicación Spring anterior	72
B.2. Snapshot de prueba anterior	72

Índice de tablas

1.1. Resumen cuantitativo del commit documentado	1
2.1. Responsabilidades principales por capa	4
3.1. Complejidad y costo conceptual de operaciones	6
3.2. Trazabilidad entre decisiones y factores McCall	7
4.1. Todos los endpoints disponibles en el backend	8
4.2. DTOs de entrada y validaciones	9
4.3. Valores enumerados permitidos	10
5.1. Usuarios configurados en memoria	11
7.1. Errores manejados por <code>GlobalExceptionHandler</code>	13
8.1. Cobertura conceptual de pruebas	14
8.2. Ponderaciones del motor de calidad	15
9.1. Distribución de archivos cambiados	16
9.2. Archivos activos documentados con código en el apéndice	16

Índice de figuras

2.1. Lectura visual de la arquitectura hexagonal aplicada al backend.	3
---	---

Capítulo 1

Resumen Ejecutivo

Qué se construyó

Se inicializó un backend empresarial para la cafetería Don Gato, orientado a gestión de inventario, productos, categorías y movimientos de stock. La implementación usa Spring Boot, JPA, validaciones Jakarta, seguridad con JWT, arquitectura hexagonal y un bloque de SQA basado en el modelo de calidad de McCall.

El commit transforma el proyecto base en una API REST completa. El núcleo del sistema ya no depende de detalles de infraestructura: el dominio contiene las reglas del negocio, la capa de aplicación coordina casos de uso, los puertos expresan contratos, la infraestructura implementa persistencia y seguridad, y la capa REST expone endpoints validados para consumidores externos.

El resultado tiene una intención clara de calidad: proteger las operaciones de escritura con autenticación, impedir inventario negativo, dejar trazabilidad de cada ajuste de stock, centralizar errores y probar reglas críticas con JUnit. La arquitectura separa responsabilidades y facilita cambiar PostgreSQL/JPA, controladores o seguridad sin tocar el corazón del negocio.

1.1 Datos del Commit

Tabla 1.1: Resumen cuantitativo del commit documentado

Hash	691fa0d
Mensaje	feat: initialize inventory management backend project with hexagonal architecture and security configuration
Archivos cambiados	66
Inserciones	3129
Eliminaciones	64
Capas creadas	Dominio, aplicación, infraestructura, interfaces REST, SQA y pruebas
Endpoints REST	17 rutas documentadas

1.2 Objetivo Funcional

El sistema permite administrar:

- Categorías de productos de la cafetería.

- Productos con precio, stock, estado y categoría.
- Movimientos de inventario de tipo ENTRADA o SALIDA.
- Login con JWT para proteger operaciones que modifican datos.
- Respuestas de error consistentes para validación, negocio y autenticación.

Capítulo 2

Arquitectura Hexagonal

2.1 Idea Central

La arquitectura hexagonal, también conocida como puertos y adaptadores, protege el núcleo del negocio. En este proyecto, el dominio no conoce Spring, JPA, HTTP ni JWT. Las dependencias apuntan hacia adentro: los controladores llaman casos de uso, los casos de uso dependen de puertos, y la infraestructura implementa esos puertos.

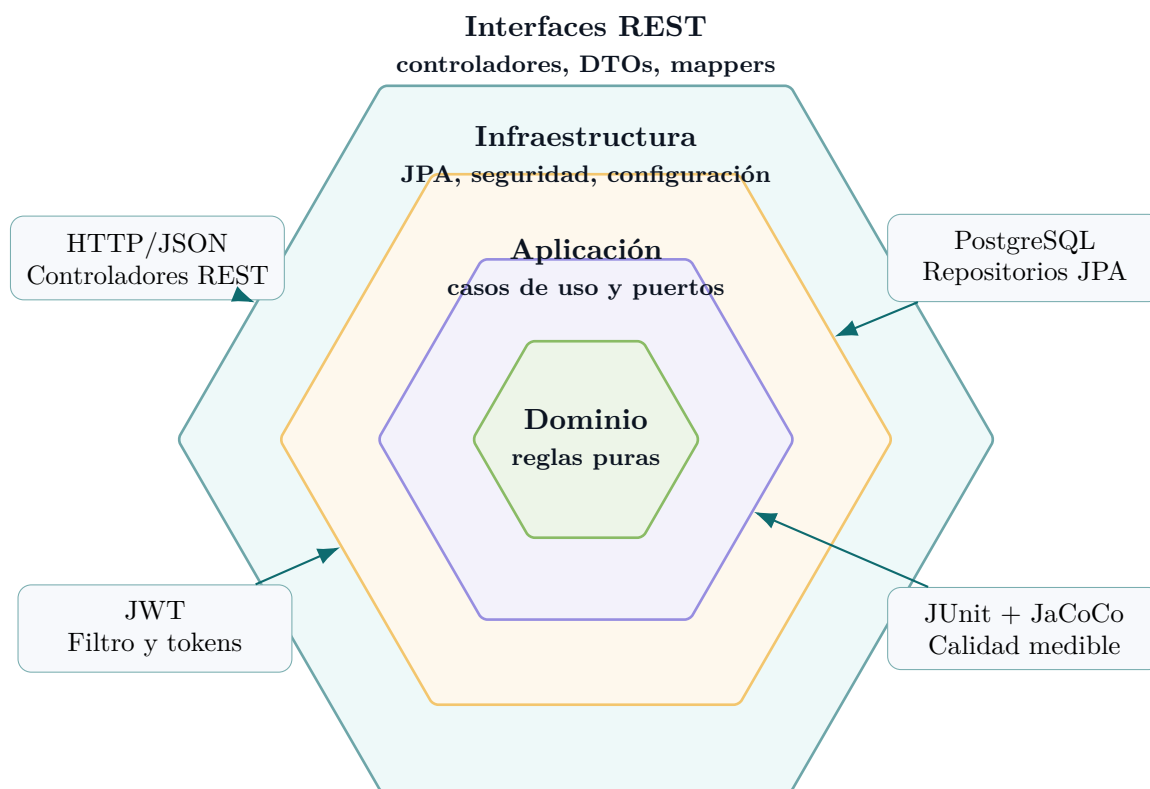


Figura 2.1: Lectura visual de la arquitectura hexagonal aplicada al backend.

2.2 Capas y Responsabilidades

Tabla 2.1: Responsabilidades principales por capa

Capa	Paquetes	Qué resuelve
Dominio	<code>domain.model</code> , <code>domain.exception</code>	Modela categorías, productos, movimientos, estados y reglas como stock no negativo, cantidades positivas y errores de negocio.
Aplicación	<code>application.usecases</code> , <code>application.ports.out</code>	Coordina transacciones, valida existencia de entidades, aplica reglas de inventario y habla con repositorios mediante puertos.
Infraestructura	<code>infrastructure.persistence</code> , <code>infrastructure.security</code> , <code>infrastructure.config</code>	Implementa persistencia JPA, entidades, mappers, seguridad JWT, CORS, usuarios en memoria y datos semilla.
Interfaces REST	<code>interfaces.rest.controller</code> , <code>mapper</code> , <code>exception</code>	dto, Expone la API HTTP, valida entradas, traduce DTOs y devuelve errores consistentes.
SQA	<code>sqa</code> , <code>src/test</code> , <code>.github/workflows</code>	Agrega motor de puntaje McCall, pruebas unitarias, JaCoCo y pipeline CI.

2.3 Flujo de Dependencias

Regla de dependencia

Las capas externas pueden conocer capas internas, pero las internas no deben conocer los detalles externos. Por eso `ProductUseCase` depende de `ProductRepositoryPort`, no de `JpaProductRepository`.

Ejemplo: caso de uso depende de puertos, no de JPA

```

1  @Service
2  @RequiredArgsConstructor
3  @Slf4j
4  @Transactional(readOnly = true)
5  public class ProductUseCase {
6
7      private final ProductRepositoryPort productRepository;
8      private final CategoryRepositoryPort categoryRepository;
9
10     @Transactional
11     public Product create(Product product) {
12         product.validate();
13         validateCategoryExists(product.getCategoryId());
14         product.calculateStatus();
15         product.setCreatedAt( LocalDateTime.now());
16         return productRepository.save(product);
17     }
18 }

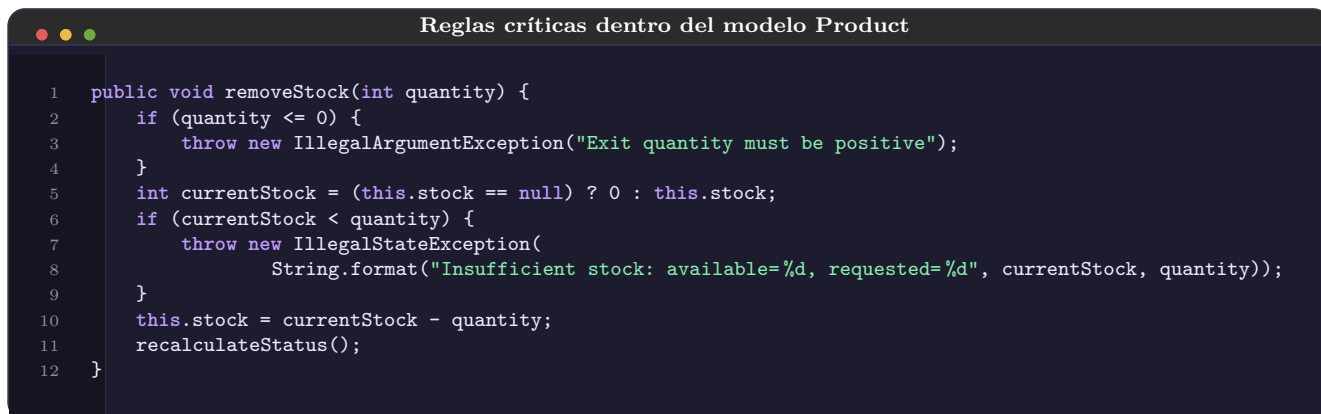
```

Capítulo 3

Complejidad Técnica y Decisiones

3.1 Dominio Rico

El dominio no se limita a contener datos. En **Product** existen operaciones de negocio: recalculando estado, agregar stock, quitar stock y validar invariantes. Esta decisión reduce errores porque las reglas viajan con el objeto central del inventario.



Reglas críticas dentro del modelo Product

```
1 public void removeStock(int quantity) {
2     if (quantity <= 0) {
3         throw new IllegalArgumentException("Exit quantity must be positive");
4     }
5     int currentStock = (this.stock == null) ? 0 : this.stock;
6     if (currentStock < quantity) {
7         throw new IllegalStateException(
8             String.format("Insufficient stock: available=%d, requested=%d", currentStock, quantity));
9     }
10    this.stock = currentStock - quantity;
11    recalculateStatus();
12 }
```

3.2 Inventario Auditable

La actualización de stock no se hace directamente desde el endpoint de productos. El commit fuerza que el cambio de inventario pase por **StockMovementUseCase**. Así cada entrada o salida queda registrada con producto, cantidad, tipo, razón y fecha.

Decisión de integridad

El stock es una consecuencia de movimientos auditable. Esta es una decisión más segura que permitir edición libre de stock en `PUT /products/{id}`.

3.3 Complejidad Operacional

Tabla 3.1: Complejidad y costo conceptual de operaciones

Operación	Complejidad lógica	Comentario
Validar producto/categoría/movimiento	$O(1)$	Revisa campos simples y reglas locales.
Crear producto	$O(1) + \text{I/O DB}$	Valida categoría, calcula estado, persiste. El costo real depende de latencia de base de datos.
Actualizar producto	$O(1) + \text{I/O DB}$	Busca por ID, valida campos, no permite cambiar stock directo.
Registrar movimiento	$O(1) + \text{I/O DB}$	Busca producto, aplica entrada/salida, guarda producto y movimiento dentro de transacción.
Listar productos/categorías/movimientos	$O(n)$	Retorna todos los registros; para producción convendría paginación.
Buscar productos por nombre	$O(n)$ o mejor con índice	La complejidad final depende del plan SQL y de índices disponibles.
Validar token JWT	$O(1)$ respecto al tamaño del sistema	Verifica firma HMAC y extrae usuario; luego carga detalles del usuario en memoria.

3.4 Transacciones

Los casos de uso están anotados con `@Transactional`. Las lecturas usan `readOnly=true`; las escrituras abren transacciones completas. En movimientos de stock, producto y movimiento se guardan como una unidad lógica: si falla la salida por stock insuficiente, no se debe persistir un movimiento incorrecto.

3.5 Modelo de Calidad de McCall

Tabla 3.2: Trazabilidad entre decisiones y factores McCall

Factor	Implementación	Evidencia
Correctness	Validaciones de dominio y DTOs	<code>Product.validate</code> , <code>StockMovement.validate</code> , Jakarta Validation
Integrity	JWT, stock auditable, errores sin stack-trace	<code>SecurityConfig</code> , <code>JwtAuthenticationFilter</code> , <code>GlobalExceptionHandler</code>
Testability	Pruebas aisladas de dominio y casos de uso	<code>ProductTest</code> , <code>StockMovementUseCaseTest</code> , JUnit
Maintainability	Capas separadas y mappers explícitos	REST mapper, persistence mapper, puertos de repositorio
Portability	Docker Compose y JPA	PostgreSQL containerizado, entidades ORM
Interoperability	API REST JSON	Controladores, DTOs y códigos HTTP claros

Capítulo 4

API REST y Endpoints

4.1 Reglas Generales

Base URL

Servidor local por defecto: `http://localhost:8080`. Todas las rutas públicas y protegidas están bajo `/api/v1`.

La seguridad configurada permite acceso público a `/api/v1/auth/**`, a lecturas de categorías y a lecturas de productos. El resto de operaciones requiere header:

Header requerido para operaciones protegidas

```
1 Authorization: Bearer <token-jwt>
```

4.2 Tabla Completa de Endpoints

Tabla 4.1: Todos los endpoints disponibles en el backend

Método	Endpoint	Seguridad	Uso
POST	<code>/api/v1/auth/login</code>	Público	Autentica usuario y devuelve JWT. Body: <code>AuthRequestDTO</code> . Respuesta: <code>AuthResponseDTO</code> .
GET	<code>/api/v1/categories</code>	Público	Lista todas las categorías. Respuesta: lista de <code>CategoryResponseDTO</code> .
GET	<code>/api/v1/categories/{id}</code>	Público	Obtiene categoría por ID. Devuelve 404 si no existe.
POST	<code>/api/v1/categories</code>	JWT requerido	Crea categoría. Body: <code>CategoryCreatedDTO</code> . Respuesta 201.
PUT	<code>/api/v1/categories/{id}</code>	JWT requerido	Actualiza categoría completa. Body: <code>CategoryCreatedDTO</code> .
DELETE	<code>/api/v1/categories/{id}</code>	JWT requerido	Elimina categoría por ID. Respuesta 204.

Método	Endpoint	Seguridad	Uso
GET	/api/v1/products	Público	Lista todos los productos. Respuesta: lista de ProductResponseDTO .
GET	/api/v1/products/{id}	Público	Obtiene producto por ID. Devuelve 404 si no existe.
GET	/api/v1/products/category/{categoryId}	Público	Lista productos asociados a una categoría existente.
GET	/api/v1/products/status/{status}	Público	Filtra por estado. Valores: AVAILABLE , OUT_OF_STOCK .
GET	/api/v1/products/search?name={texto}	Público	Busca productos por nombre ignorando mayúsculas/minúsculas.
POST	/api/v1/products	JWT requerido	Crea producto. Body: ProductCreateDTO . Respuesta 201.
PUT	/api/v1/products/{id}	JWT requerido	Actualiza nombre, precio y categoría. No actualiza stock directo.
DELETE	/api/v1/products/{id}	JWT requerido	Elimina producto por ID. Respuesta 204.
POST	/api/v1/stock-movements	JWT requerido	Registra entrada/salida y actualiza stock. Body: StockMovementCreateDTO . Respuesta 201.
GET	/api/v1/stock-movements	JWT requerido	Lista todos los movimientos de stock.
GET	/api/v1/stock-movements/product/{productId}	JWT requerido	Lista movimientos de un producto. Devuelve 404 si el producto no existe.

4.3 Contratos de Entrada

Tabla 4.2: DTOs de entrada y validaciones

DTO	Campos	Validaciones principales
AuthRequestDTO	username, password	Ambos obligatorios con @NotBlank .
CategoryCreateDTO	name, description	name obligatorio y máximo 100 caracteres; description máximo 255.
ProductCreateDTO	name, price, stock, categoryId	Nombre obligatorio, precio no negativo, stock inicial no negativo, categoría obligatoria.
ProductUpdateDTO	name, price, categoryId	Nombre y precio obligatorios; categoría opcional pero si llega debe existir.
StockMovementCreateDTO	productId, type, quantity, reason	Producto, tipo y razón obligatorios; cantidad mínima 1.

4.4 Enums Expuestos

Tabla 4.3: Valores enumerados permitidos

Enum	Valores
ProductStatus	AVAILABLE, OUT_OF_STOCK
MovementType	ENTRADA, SALIDA
MovementReason	VENTA, REPOSICION, AJUSTE, DEVOLUCION, MERMA

4.5 Ejemplos de Uso

Login

```
1 curl -X POST http://localhost:8080/api/v1/auth/login \  
2 -H "Content-Type: application/json" \  
3 -d "{\"username\":\"admin\",\"password\":\"dongato2026\"}"
```

Crear producto

```
1 curl -X POST http://localhost:8080/api/v1/products \  
2 -H "Content-Type: application/json" \  
3 -H "Authorization: Bearer <token>" \  
4 -d "{\"name\":\"Latte\",\"price\":3.50,\"stock\":20,\"categoryId\":1}"
```

Registrar salida de stock

```
1 curl -X POST http://localhost:8080/api/v1/stock-movements \  
2 -H "Content-Type: application/json" \  
3 -H "Authorization: Bearer <token>" \  
4 -d "{\"productId\":1,\"type\":\"SALIDA\",\"quantity\":2,\"reason\":\"VENTA\"}"
```

Capítulo 5

Seguridad

5.1 Autenticación JWT

El backend usa autenticación stateless. El usuario llama `/api/v1/auth/login`, Spring Security valida credenciales en memoria y `JwtTokenProvider` genera un token firmado con HMAC-SHA256. En cada request posterior, `JwtAuthenticationFilter` extrae el header `Authorization`, valida el token y registra la autenticación en el `SecurityContext`.

```
Reglas de autorización

1 .authorizeHttpRequests(auth -> auth
2     .requestMatchers("/api/v1/auth/**").permitAll()
3     .requestMatchers(HttpMethod.GET, "/api/v1/categories/**").permitAll()
4     .requestMatchers(HttpMethod.GET, "/api/v1/products/**").permitAll()
5     .anyRequest().authenticated()
6 )
```

5.2 Usuarios de Prueba

Tabla 5.1: Usuarios configurados en memoria

Usuario	Contraseña	Rol
admin	dongato2026	ADMIN
cajero	cafe123	USER

Nota de endurecimiento

Para producción, estos usuarios deben migrarse a una tabla de usuarios, la clave JWT debe vivir en variables de entorno o gestor de secretos, y se debería agregar refresh token, rotación de claves, políticas de expiración y auditoría de intentos fallidos.

5.3 CORS y Sesiones

La API está preparada para frontends locales en `http://localhost:3000` y `http://localhost:5173`. La política usa sesiones stateless y desactiva CSRF porque la API no depende de cookies de sesión.

Capítulo 6

Persistencia y Datos

6.1 Modelo Persistente

La infraestructura contiene entidades JPA separadas de los modelos de dominio. Esta duplicación es intencional: el dominio representa reglas y el modelo persistente representa tablas. Los mappers traducen entre ambos mundos.

Por qué usar mappers

Los mappers evitan que anotaciones ORM y detalles de base de datos contaminen los modelos de negocio. Esto mejora mantenibilidad y portabilidad.

6.2 Repositorios

Los repositorios JPA implementan consultas derivadas como:

- Buscar productos por categoría.
- Buscar productos por estado.
- Buscar por nombre ignorando mayúsculas/minúsculas.
- Listar movimientos por producto.

6.3 Docker Compose

El servicio PostgreSQL queda configurado con imagen `postgres:16-alpine`, base `dongato_db`, usuario `dongato_user`, volumen persistente y healthcheck con `pg_isready`.

Capítulo 7

Manejo de Errores

7.1 Formato Común

Todos los errores se empaquetan en `ApiErrorDTO`, con estado HTTP, nombre del error, mensaje, ruta, timestamp y errores de campo cuando aplica.

Respuesta de error estandarizada

```
1 ApiErrorDTO error = ApiErrorDTO.builder()
2     .status(status.value())
3     .error(status.getReasonPhrase())
4     .message(message)
5     .path(request.getRequestURI())
6     .timestamp(LocalDateTime.now())
7     .build();
```

7.2 Mapa de Excepciones

Tabla 7.1: Errores manejados por `GlobalExceptionHandler`

Excepción	HTTP	Motivo
<code>ResourceNotFoundException</code>	404	Recurso inexistente.
<code>BusinessRuleException</code>	409	Violación de regla de negocio, como stock insuficiente.
<code>IllegalArgumentException</code>	400	Argumento inválido en dominio o aplicación.
<code>MethodArgumentNotValidException</code>	400	Validación Jakarta fallida en DTO.
<code>BadCredentialsException</code>	401	Usuario o contraseña inválidos.
<code>Exception</code>	500	Error inesperado sin filtrar detalles internos.

Capítulo 8

Pruebas, SQA y CI/CD

8.1 Pruebas Implementadas

El commit agrega pruebas unitarias para dominio, casos de uso y motor de calidad. Las reglas más sensibles quedan verificadas sin depender de HTTP ni de base de datos real.

Tabla 8.1: Cobertura conceptual de pruebas

Archivo de prueba	Qué verifica
<code>ProductTest</code>	Validación de producto, entradas, salidas, stock insuficiente y cálculo automático de estado.
<code>CategoryTest</code>	Invariantes básicas de categoría.
<code>StockMovementTest</code>	Datos obligatorios y coherencia del movimiento.
<code>CategoryUseCaseTest</code>	Creación, consulta, actualización y eliminación con repositorio simulado.
<code>StockMovementUseCaseTest</code>	Entradas/salidas, persistencia de movimiento y error por stock insuficiente.
<code>QualityScoringEngineTest</code>	Fórmula de puntaje, penalizaciones y rangos de calificación.
<code>DonGatoInventoryApplicationTest</code>	Carga del contexto Spring.

8.2 Motor de Calidad McCall

El motor calcula un puntaje de 0 a 100 con ponderaciones:

$$\text{score} = 0,30(\text{testability}) + 0,30(\text{correctness}) + 0,20(\text{maintainability}) + 0,20(\text{integrity})$$

8.3 Pipeline

El workflow `ci-cd.yml` ejecuta `checkout`, prepara JDK, corre `mvn clean verify`, genera JaCoCo y sube reportes de cobertura y pruebas como artefactos.

Tabla 8.2: Ponderaciones del motor de calidad

Factor	Peso	Métrica usada
Correctness	30 %	Bugs detectados.
Testability	30 %	Porcentaje de cobertura.
Maintainability	20 %	Code smells.
Integrity	20 %	Vulnerabilidades; tolerancia cero.

Observación técnica

El `pom.xml` declara `java.version=25`, mientras el workflow configura JDK 21. Conviene alinear ambas versiones antes de una entrega formal para evitar diferencias entre entorno local y CI.

Capítulo 9

Inventario de Archivos del Commit

9.1 Resumen por Tipo de Cambio

Tabla 9.1: Distribución de archivos cambiados

Tipo	Cantidad	Detalle
Agregados	59	Nuevo backend por capas, DTOs, seguridad, SQA y tests.
Modificados	5	Pipeline, .gitignore, Docker Compose, pom.xml, propiedades.
Eliminados	2	Clases base del paquete anterior com.sqa_inventory.backend.
Total	66	Alcance completo del commit 691fa0d.

9.2 Lista de Archivos

Tabla 9.2: Archivos activos documentados con código en el apéndice

Tipo	Ruta
M	.github/workflows/ci-cd.yml
M	.gitignore
M	backend/compose.yaml
M	backend/pom.xml
A	backend/src/main/java/com/dongato/inventory/DonGatoInventoryApplication.java
A	backend/src/main/java/com/dongato/inventory/application/ports/out/CategoryRepositoryPort.java
A	backend/src/main/java/com/dongato/inventory/application/ports/out/ProductRepositoryPort.java
A	backend/src/main/java/com/dongato/inventory/application/ports/out/StockMovementRepositoryPort.java
A	backend/src/main/java/com/dongato/inventory/application/usecases/CategoryUseCase.java
A	backend/src/main/java/com/dongato/inventory/application/usecases/ProductUseCase.java

Tipo	Ruta
A	backend/src/main/java/com/dongato/inventory/application/usecases/StockMovementUseCase.java
A	backend/src/main/java/com/dongato/inventory/domain/exception/BusinessRuleException.java
A	backend/src/main/java/com/dongato/inventory/domain/exception/InsufficientStockException.java
A	backend/src/main/java/com/dongato/inventory/domain/exception/ResourceNotFoundException.java
A	backend/src/main/java/com/dongato/inventory/domain/model/Category.java
A	backend/src/main/java/com/dongato/inventory/domain/model/MovementReason.java
A	backend/src/main/java/com/dongato/inventory/domain/model/MovementType.java
A	backend/src/main/java/com/dongato/inventory/domain/model/Product.java
A	backend/src/main/java/com/dongato/inventory/domain/model/ProductStatus.java
A	backend/src/main/java/com/dongato/inventory/domain/model/StockMovement.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/config/DataSeeder.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapters/CategoryRepositoryAdapter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapters/ProductRepositoryAdapter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapters/StockMovementRepositoryAdapter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entities/CategoryEntity.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entities/ProductEntity.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entities/StockMovementEntity.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mappers/CategoryPersistenceMapper.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mappers/ProductPersistenceMapper.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mappers/StockMovementPersistenceMapper.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repositories/JpaCategoryRepository.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repositories/JpaProductRepository.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repositories/JpaStockMovementRepository.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtAuthenticationFilter.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtTokenProvider.java
A	backend/src/main/java/com/dongato/inventory/infrastructure/security/SecurityConfig.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/AuthController.java

Tipo	Ruta
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/CategoryController.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/ProductController.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/StockMovementController.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ApiErrorDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthRequestDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryCreatedDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductCreateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductUpdateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementCreateDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementResponseDTO.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/exception/GlobalExceptionHandler.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/CategoryRestMapper.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/ProductRestMapper.java
A	backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/StockMovementRestMapper.java
A	backend/src/main/java/com/dongato/inventory/sqa/QualityScoringEngine.java
M	backend/src/main/resources/application.properties
A	backend/src/test/java/com/dongato/inventory/DonGatoInventoryApplicationTests.java
A	backend/src/test/java/com/dongato/inventory/application/usecases/CategoryUseCaseTest.java
A	backend/src/test/java/com/dongato/inventory/application/usecases/StockMovementUseCaseTest.java
A	backend/src/test/java/com/dongato/inventory/domain/model/CategoryTest.java
A	backend/src/test/java/com/dongato/inventory/domain/model/ProductTest.java
A	backend/src/test/java/com/dongato/inventory/domain/model/StockMovementTest.java
A	backend/src/test/java/com/dongato/inventory/sqa/QualityScoringEngineTest.java
A	backend/src/test/resources/application-test.properties

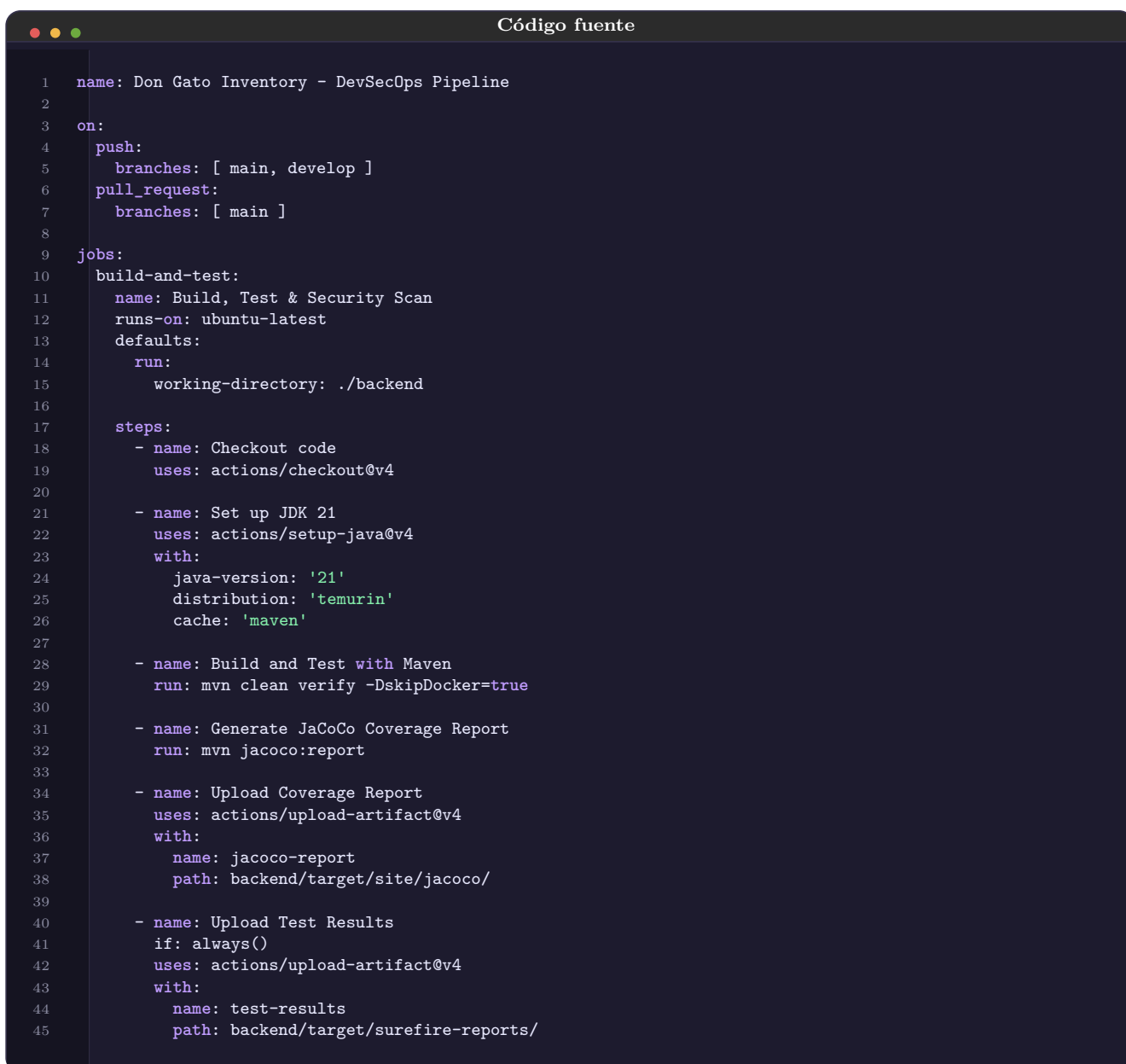
Los dos archivos eliminados se incluyen como snapshot textual al final del apéndice para conservar la trazabilidad del total de 66 cambios.

Apéndice A

Código Completo de los Archivos Activos

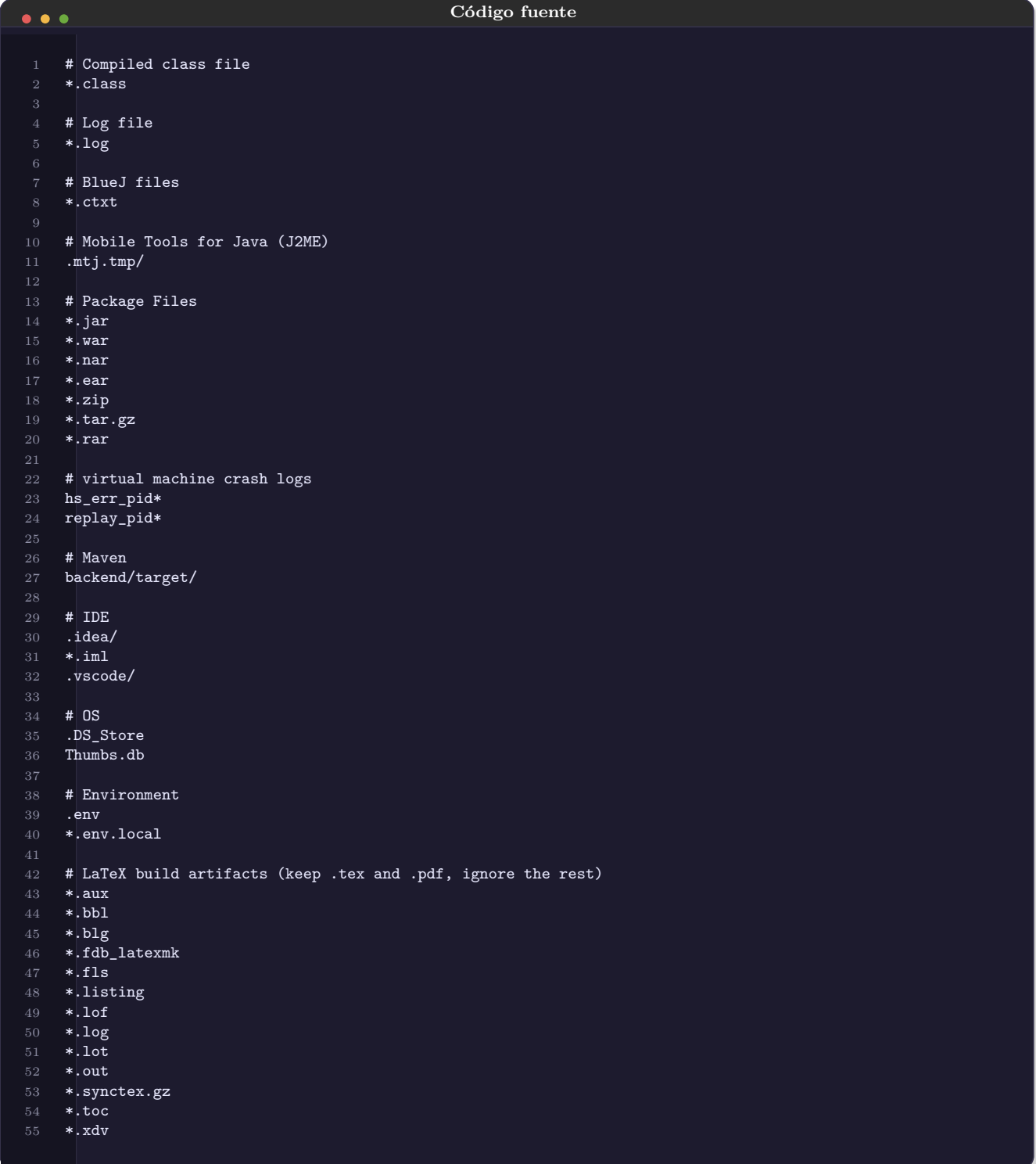
A.1 Configuración de Proyecto

`.github/workflows/ci-cd.yml`

A screenshot of a code editor window titled "Código fuente". The editor displays a GitHub Actions workflow file named ".github/workflows/ci-cd.yml". The code is written in YAML and defines a pipeline for building, testing, and deploying code. It includes triggers for push and pull requests, a job named "build-and-test" that runs on "ubuntu-latest", and a series of steps: checking out code, setting up JDK 21, building and testing with Maven, generating a JaCoCo coverage report, uploading the coverage report, and uploading test results. The workflow is named "Don Gato Inventory - DevSecOps Pipeline".

```
1 name: Don Gato Inventory - DevSecOps Pipeline
2
3 on:
4   push:
5     branches: [ main, develop ]
6   pull_request:
7     branches: [ main ]
8
9 jobs:
10  build-and-test:
11    name: Build, Test & Security Scan
12    runs-on: ubuntu-latest
13    defaults:
14      run:
15        working-directory: ./backend
16
17    steps:
18      - name: Checkout code
19        uses: actions/checkout@v4
20
21      - name: Set up JDK 21
22        uses: actions/setup-java@v4
23        with:
24          java-version: '21'
25          distribution: 'temurin'
26          cache: 'maven'
27
28      - name: Build and Test with Maven
29        run: mvn clean verify -DskipDocker=true
30
31      - name: Generate JaCoCo Coverage Report
32        run: mvn jacoco:report
33
34      - name: Upload Coverage Report
35        uses: actions/upload-artifact@v4
36        with:
37          name: jacoco-report
38          path: backend/target/site/jacoco/
39
40      - name: Upload Test Results
41        if: always()
42        uses: actions/upload-artifact@v4
43        with:
44          name: test-results
45          path: backend/target/surefire-reports/
```


.gitignore



```
1  # Compiled class file
2  *.class
3
4  # Log file
5  *.log
6
7  # BlueJ files
8  *.ctxt
9
10 # Mobile Tools for Java (J2ME)
11 .mtj.tmp/
12
13 # Package Files
14 *.jar
15 *.war
16 *.nar
17 *.ear
18 *.zip
19 *.tar.gz
20 *.rar
21
22 # virtual machine crash logs
23 hs_err_pid*
24 replay_pid*
25
26 # Maven
27 backend/target/
28
29 # IDE
30 .idea/
31 *.iml
32 .vscode/
33
34 # OS
35 .DS_Store
36 Thumbs.db
37
38 # Environment
39 .env
40 *.env.local
41
42 # LaTeX build artifacts (keep .tex and .pdf, ignore the rest)
43 *.aux
44 *.bbl
45 *.blg
46 *.fdb_latexmk
47 *.fls
48 *.listing
49 *.lof
50 *.log
51 *.lot
52 *.out
53 *.synctex.gz
54 *.toc
55 *.xdv
```

backend/compose.yaml



```
1  services:
2    postgres:
3      image: 'postgres:16-alpine'
4      container_name: dongato-postgres
5      environment:
6        POSTGRES_DB: dongato_db
```

```

7     POSTGRES_USER: dongato_user
8     POSTGRES_PASSWORD: dongato123
9     ports:
10      - '5432:5432'
11     volumes:
12      - dongato_data:/var/lib/postgresql/data
13     healthcheck:
14      test: ["CMD-SHELL", "pg_isready -U dongato_user -d dongato_db"]
15      interval: 10s
16      timeout: 5s
17      retries: 5
18     restart: unless-stopped
19
20 volumes:
21     dongato_data:
22     driver: local

```

backend/pom.xml

Código fuente

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4    <modelVersion>4.0.0</modelVersion>
5    <parent>
6      <groupId>org.springframework.boot</groupId>
7      <artifactId>spring-boot-starter-parent</artifactId>
8      <version>4.0.6</version>
9      <relativePath/> <!-- lookup parent from repository -->
10   </parent>
11   <groupId>com.dongato</groupId>
12   <artifactId>inventory</artifactId>
13   <version>1.0.0-SNAPSHOT</version>
14   <name>dongato-inventory</name>
15   <description>Cafeteria Don Gato - Inventory Management System (SQA Project)</description>
16
17   <properties>
18     <java.version>25</java.version>
19     <jjwt.version>0.12.6</jjwt.version>
20   </properties>
21
22   <dependencies>
23     <!-- Web & REST (Interoperability) -->
24     <dependency>
25       <groupId>org.springframework.boot</groupId>
26       <artifactId>spring-boot-starter-webmvc</artifactId>
27     </dependency>
28
29     <!-- Persistence / JPA (Integrity, Reliability) -->
30     <dependency>
31       <groupId>org.springframework.boot</groupId>
32       <artifactId>spring-boot-starter-data-jpa</artifactId>
33     </dependency>
34
35     <!-- Validation (Correctness) -->
36     <dependency>
37       <groupId>org.springframework.boot</groupId>
38       <artifactId>spring-boot-starter-validation</artifactId>
39     </dependency>
40
41     <!-- Security (Integrity) -->
42     <dependency>
43       <groupId>org.springframework.boot</groupId>
44       <artifactId>spring-boot-starter-security</artifactId>
45     </dependency>
46
47     <!-- JWT (Integrity - Authentication) -->
48     <dependency>
49       <groupId>io.jsonwebtoken</groupId>
50       <artifactId>jjwt-api</artifactId>

```

```

51     <version>${jwt version}</version>
52 </dependency>
53 <dependency>
54     <groupId>io.jsonwebtoken</groupId>
55     <artifactId>jjwt-impl</artifactId>
56     <version>${jwt version}</version>
57     <scope>runtime</scope>
58 </dependency>
59 <dependency>
60     <groupId>io.jsonwebtoken</groupId>
61     <artifactId>jjwt-jackson</artifactId>
62     <version>${jwt version}</version>
63     <scope>runtime</scope>
64 </dependency>
65
66 <!-- Docker Compose integration (Portability) -->
67 <dependency>
68     <groupId>org.springframework.boot</groupId>
69     <artifactId>spring-boot-docker-compose</artifactId>
70     <scope>runtime</scope>
71     <optional>true</optional>
72 </dependency>
73
74 <!-- PostgreSQL driver -->
75 <dependency>
76     <groupId>org.postgresql</groupId>
77     <artifactId>postgresql</artifactId>
78     <scope>runtime</scope>
79 </dependency>
80
81 <!-- DevTools (Maintainability) -->
82 <dependency>
83     <groupId>org.springframework.boot</groupId>
84     <artifactId>spring-boot-devtools</artifactId>
85     <scope>runtime</scope>
86     <optional>true</optional>
87 </dependency>
88
89 <!-- Lombok (Maintainability) -->
90 <dependency>
91     <groupId>org.projectlombok</groupId>
92     <artifactId>lombok</artifactId>
93     <optional>true</optional>
94 </dependency>
95
96 <!-- ===== TEST DEPENDENCIES ===== -->
97 <dependency>
98     <groupId>org.springframework.boot</groupId>
99     <artifactId>spring-boot-starter-test</artifactId>
100    <scope>test</scope>
101 </dependency>
102 <dependency>
103     <groupId>org.springframework.security</groupId>
104     <artifactId>spring-security-test</artifactId>
105     <scope>test</scope>
106 </dependency>
107 <dependency>
108     <groupId>com.h2database</groupId>
109     <artifactId>h2</artifactId>
110     <scope>test</scope>
111 </dependency>
112 </dependencies>
113
114 <build>
115     <plugins>
116         <plugin>
117             <groupId>org.springframework.boot</groupId>
118             <artifactId>spring-boot-maven-plugin</artifactId>
119             <configuration>
120                 <excludes>
121                     <exclude>
122                         <groupId>org.projectlombok</groupId>
123                         <artifactId>lombok</artifactId>
124                     </exclude>
125                 </excludes>

```

```

126     </configuration>
127 </plugin>
128 <plugin>
129   <groupId>org.apache.maven.plugins</groupId>
130   <artifactId>maven-compiler-plugin</artifactId>
131   <executions>
132     <execution>
133       <id>default-compile</id>
134       <phase>compile</phase>
135       <goals>
136         <goal>compile</goal>
137       </goals>
138       <configuration>
139         <annotationProcessorPaths>
140           <path>
141             <groupId>org.projectlombok</groupId>
142             <artifactId>lombok</artifactId>
143           </path>
144         </annotationProcessorPaths>
145       </configuration>
146     </execution>
147     <execution>
148       <id>default-testCompile</id>
149       <phase>test-compile</phase>
150       <goals>
151         <goal>testCompile</goal>
152       </goals>
153       <configuration>
154         <annotationProcessorPaths>
155           <path>
156             <groupId>org.projectlombok</groupId>
157             <artifactId>lombok</artifactId>
158           </path>
159         </annotationProcessorPaths>
160       </configuration>
161     </execution>
162   </executions>
163 </plugin>
164 <!-- JaCoCo for Code Coverage (Testability) -->
165 <plugin>
166   <groupId>org.jacoco</groupId>
167   <artifactId>jacoco-maven-plugin</artifactId>
168   <version>0.8.14</version>
169   <executions>
170     <execution>
171       <goals>
172         <goal>prepare-agent</goal>
173       </goals>
174     </execution>
175     <execution>
176       <id>report</id>
177       <phase>test</phase>
178       <goals>
179         <goal>report</goal>
180       </goals>
181     </execution>
182   </executions>
183 </plugin>
184 </plugins>
185 </build>
186
187 </project>

```

backend/src/main/resources/application.properties

Código fuente

```

1  # =====
2  # Cafeteria Don Gato - Inventory Management System
3  # =====
4

```

```

5  spring application name dongato inventory
6
7  # --- Database (auto-configured by spring-boot-docker-compose) ---
8  spring jpa hibernate ddl auto update
9  spring jpa show sql false
10 spring jpa properties hibernate format_sql true
11 spring jpa properties hibernate dialect org.hibernate.dialect.PostgreSQLDialect
12 spring jpa open in view false
13
14 # --- Server ---
15 server port 8080
16 server error include message always
17 server error include binding errors always
18
19 # --- Security Headers ---
20 server servlet session cookie http only true
21 server servlet session cookie secure true
22
23 # --- Logging ---
24 logging level root INFO
25 logging level com.dongato.inventory DEBUG
26 logging level org.springframework.security INFO

```

backend/src/test/resources/application-test.properties

Código fuente

```

1  # =====
2  # Test Profile Configuration
3  # =====
4
5  spring.application.name=dongato-inventory-test
6
7  # Use H2 in-memory database for tests
8  spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false
9  spring.datasource.driver-class-name=org.h2.Driver
10 spring.datasource.username=sa
11 spring.datasource.password=
12
13 spring.jpa.hibernate.ddl-auto=create-drop
14 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
15 spring.jpa.show-sql=false
16 spring.jpa.open-in-view=false
17
18 # Disable Docker Compose in tests
19 spring.docker.compose.enabled=false
20
21 # JWT config for tests
22 jwt.secret=TestSecretKeyForJWTTokenMustBeAtLeast256BitsLongForHMACSHA
23 jwt.expiration-ms=3600000
24
25 logging.level.root=WARN

```

A.2 Aplicación

backend/src/main/java/com/dongato/inventory/DonGatoInventoryApplication.java

Código fuente

```

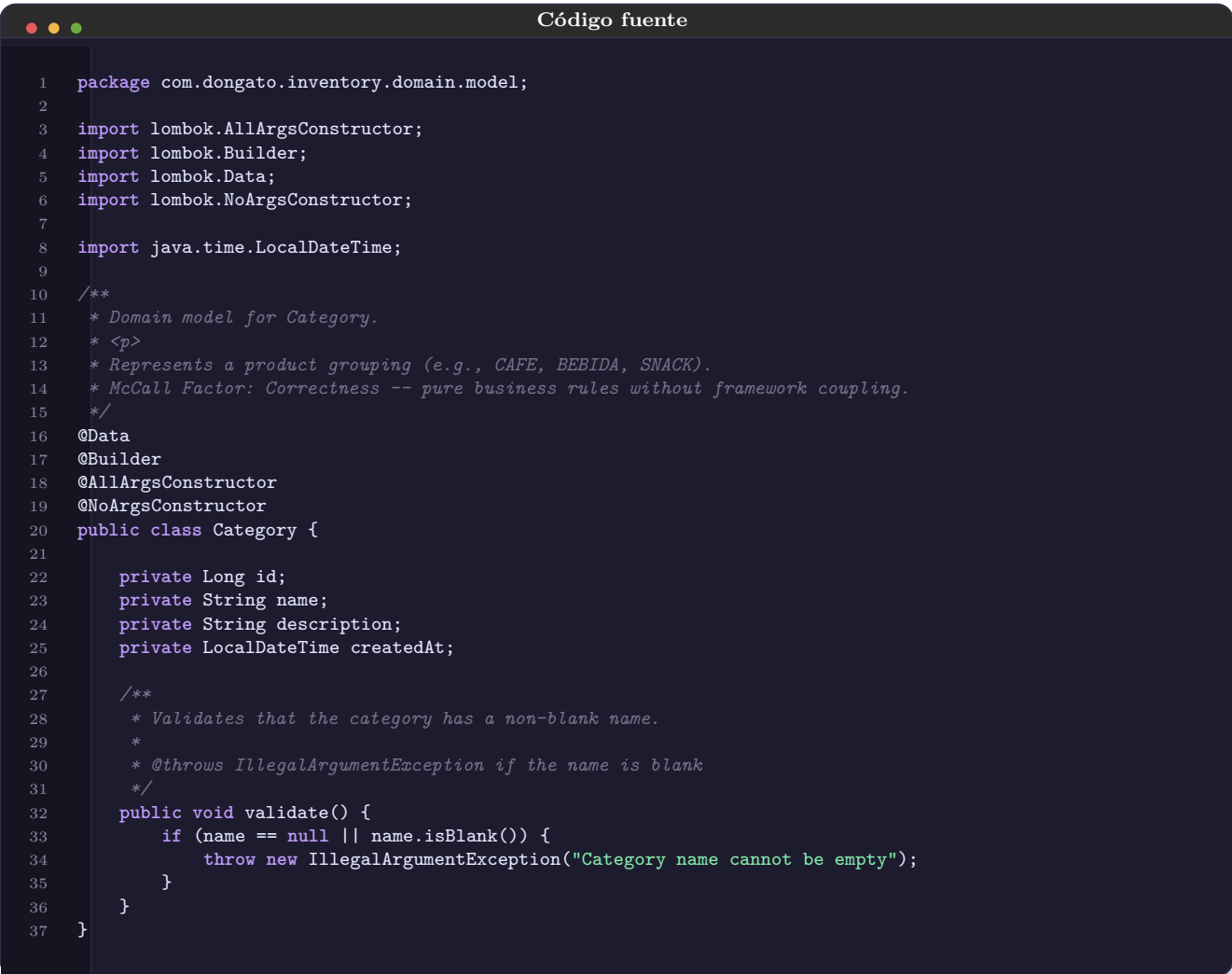
1  package com.dongato.inventory;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6  /**

```

```
7  * Cafetería Don Gato - Inventory Management System.
8  * <p>
9  * Entry point for the Spring Boot application.
10 * Uses Clean Architecture with McCall quality model traceability.
11 */
12 @SpringBootApplication
13 public class DonGatoInventoryApplication {
14
15     public static void main(String[] args) {
16         SpringApplication.run(DonGatoInventoryApplication.class, args);
17     }
18 }
```

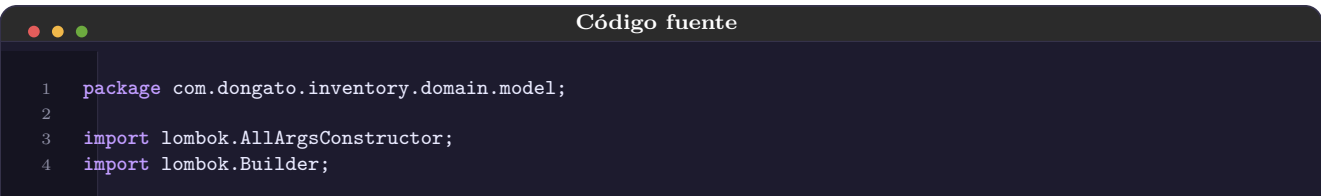
A.3 Dominio

backend/src/main/java/com/dongato/inventory/domain/model/Category.java



```
1  package com.dongato.inventory.domain.model;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Builder;
5  import lombok.Data;
6  import lombok.NoArgsConstructor;
7
8  import java.time.LocalDateTime;
9
10 /**
11  * Domain model for Category.
12  * <p>
13  * Represents a product grouping (e.g., CAFE, BEBIDA, SNACK).
14  * McCall Factor: Correctness -- pure business rules without framework coupling.
15  */
16 @Data
17 @Builder
18 @AllArgsConstructor
19 @NoArgsConstructor
20 public class Category {
21
22     private Long id;
23     private String name;
24     private String description;
25     private LocalDateTime createdAt;
26
27     /**
28      * Validates that the category has a non-blank name.
29      *
30      * @throws IllegalArgumentException if the name is blank
31      */
32     public void validate() {
33         if (name == null || name.isBlank()) {
34             throw new IllegalArgumentException("Category name cannot be empty");
35         }
36     }
37 }
```

backend/src/main/java/com/dongato/inventory/domain/model/Product.java



```
1  package com.dongato.inventory.domain.model;
2
3  import lombok.AllArgsConstructor;
4  import lombok.Builder;
```

```

5  import lombok Data
6  import lombok NoArgsConstructor;
7
8  import java.math.BigDecimal;
9  import java.time.LocalDateTime;
10
11  /**
12   * Domain model for Product -- the heart of the inventory.
13   * <p>
14   * Contains business logic for stock management and automatic status calculation.
15   * McCall Factor: Correctness -- encapsulates invariants that prevent invalid states.
16   */
17  @Data
18  @Builder
19  @AllArgsConstructor
20  @NoArgsConstructor
21  public class Product {
22
23      private Long id;
24      private String name;
25      private BigDecimal price;
26      private Integer stock;
27      private ProductStatus status;
28      private Long categoryId;
29      private LocalDateTime createdAt;
30
31      /**
32       * Automatically recalculates the product status based on current stock level.
33       * Stock <= 0 results in OUT_OF_STOCK, otherwise AVAILABLE.
34       */
35      public void recalculateStatus() {
36          this.status = (this.stock != null && this.stock > 0)
37              ? ProductStatus AVAILABLE
38              : ProductStatus OUT_OF_STOCK;
39      }
40
41      /**
42       * Applies a stock entry (adds quantity).
43       *
44       * @param quantity the quantity to add (must be positive)
45       * @throws IllegalArgumentException if quantity is not positive
46       */
47      public void addStock(int quantity) {
48          if (quantity <= 0) {
49              throw new IllegalArgumentException("Entry quantity must be positive");
50          }
51          this.stock = (this.stock == null ? 0 : this.stock) + quantity;
52          recalculateStatus();
53      }
54
55      /**
56       * Applies a stock exit (subtracts quantity).
57       *
58       * @param quantity the quantity to subtract (must be positive)
59       * @throws IllegalArgumentException if quantity is not positive
60       * @throws IllegalStateException if insufficient stock
61       */
62      public void removeStock(int quantity) {
63          if (quantity <= 0) {
64              throw new IllegalArgumentException("Exit quantity must be positive");
65          }
66          int currentStock = (this.stock == null) ? 0 : this.stock;
67          if (currentStock < quantity) {
68              throw new IllegalStateException(
69                  String.format("Insufficient stock: available=%d, requested=%d", currentStock, quantity)
70              );
71          }
72          this.stock = currentStock - quantity;
73          recalculateStatus();
74      }
75
76      /**
77       * Validates basic product invariants.
78       */
79      public void validate() {

```

```
79         if (name == null || name.isBlank()) {
80             throw new IllegalArgumentException("Product name cannot be empty");
81         }
82         if (price == null || price.compareTo(BigDecimal.ZERO) < 0) {
83             throw new IllegalArgumentException("Product price must be non-negative");
84         }
85         if (stock != null && stock < 0) {
86             throw new IllegalArgumentException("Product stock cannot be negative");
87         }
88     }
89 }
```

backend/src/main/java/com/dongato/inventory/domain/model/ProductStatus.java

Código fuente

```
1 package com.dongato.inventory.domain.model;
2
3 /**
4  * Represents the status of a product in the inventory.
5  * <p>
6  * McCall Factor: Correctness -- ensures valid product states.
7  */
8 public enum ProductStatus {
9     AVAILABLE,
10    OUT_OF_STOCK
11 }
```

backend/src/main/java/com/dongato/inventory/domain/model/StockMovement.java

Código fuente

```
1 package com.dongato.inventory.domain.model;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Builder;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7
8 import java.time.LocalDateTime;
9
10 /**
11  * Domain model for StockMovement -- audit trail for inventory changes.
12  * <p>
13  * Records every stock change with type (ENTRADA/SALIDA) and reason.
14  * McCall Factor: Integrity -- enables full traceability of inventory operations.
15  */
16 @Data
17 @Builder
18 @AllArgsConstructor
19 @NoArgsConstructor
20 public class StockMovement {
21
22     private Long id;
23     private Long productId;
24     private MovementType type;
25     private Integer quantity;
26     private MovementReason reason;
27     private LocalDateTime createdAt;
28
29     /**
30      * Validates that the movement has coherent data.
31      */
32     public void validate() {
33         if (productId == null) {
34             throw new IllegalArgumentException("Product ID is required for stock movement");
35         }
36     }
37 }
```



```
36         if (type == null) {
37             throw new IllegalArgumentException("Movement type is required");
38         }
39         if (quantity == null || quantity <= 0) {
40             throw new IllegalArgumentException("Movement quantity must be positive");
41         }
42         if (reason == null) {
43             throw new IllegalArgumentException("Movement reason is required");
44         }
45     }
46 }
```

backend/src/main/java/com/dongato/inventory/domain/model/MovementType.java

Código fuente

```
1 package com.dongato.inventory.domain.model;
2
3 /**
4  * Type of stock movement.
5  * <p>
6  * McCall Factor: Integrity -- ensures auditability of inventory changes.
7  */
8 public enum MovementType {
9     ENTRADA,
10    SALIDA
11 }
```

backend/src/main/java/com/dongato/inventory/domain/model/MovementReason.java

Código fuente

```
1 package com.dongato.inventory.domain.model;
2
3 /**
4  * Reason for a stock movement.
5  * <p>
6  * McCall Factor: Integrity -- provides context for audit trail.
7  */
8 public enum MovementReason {
9     VENTA,
10    REPOSICION,
11    AJUSTE,
12    DEVOLUCION,
13    MERMA
14 }
```

backend/src/main/java/com/dongato/inventory/domain/exception/BusinessRuleException.java

Código fuente

```
1 package com.dongato.inventory.domain.exception;
2
3 /**
4  * Thrown when a business rule is violated.
5  * McCall Factor: Correctness -- enforces business invariants at domain level.
6  */
7 public class BusinessRuleException extends RuntimeException {
8
9     public BusinessRuleException(String message) {
10         super(message);
11     }
12 }
```

```
12 }
```

backend/src/main/java/com/dongato/inventory/domain/exception/InsufficientStockException.java

Código fuente

```
1 package com.dongato.inventory.domain.exception;
2
3 /**
4  * Thrown when there is insufficient stock for an operation.
5  * McCall Factor: Correctness -- prevents inventory going negative.
6  */
7 public class InsufficientStockException extends BusinessException {
8
9     public InsufficientStockException(String productName, int available, int requested) {
10         super(String.format("Insufficient stock for '%s': available=%d, requested=%d",
11             productName, available, requested));
12     }
13 }
```

backend/src/main/java/com/dongato/inventory/domain/exception/ResourceNotFoundException.java

Código fuente

```
1 package com.dongato.inventory.domain.exception;
2
3 /**
4  * Thrown when a requested resource is not found.
5  * McCall Factor: Correctness -- clear error semantics.
6  */
7 public class ResourceNotFoundException extends RuntimeException {
8
9     public ResourceNotFoundException(String resourceName, Long id) {
10         super(String.format("%s not found with id: %d", resourceName, id));
11     }
12
13     public ResourceNotFoundException(String message) {
14         super(message);
15     }
16 }
```

A.4 Aplicación: Puertos y Casos de Uso

backend/src/main/java/com/dongato/inventory/application/ports/out/CategoryRepositoryPort.java

Código fuente

```
1 package com.dongato.inventory.application.ports.out;
2
3 import com.dongato.inventory.domain.model.Category;
4
5 import java.util.List;
6 import java.util.Optional;
7
8 /**
9  * Output port for Category persistence.
10  * <p>
11  * McCall Factor: Reusability -- abstracts persistence from business logic (DIP).
12  */
13 public interface CategoryRepositoryPort {
```

```
14
15     Category save(Category category);
16
17     Optional<Category> findById(Long id);
18
19     List<Category> findAll();
20
21     boolean existsById(Long id);
22
23     boolean existsByName(String name);
24
25     void deleteById(Long id);
26 }
```

backend/src/main/java/com/dongato/inventory/application/ports/out/ProductRepositoryPort.java

Código fuente

```
1 package com.dongato.inventory.application.ports.out;
2
3 import com.dongato.inventory.domain.model.Product;
4 import com.dongato.inventory.domain.model.ProductStatus;
5
6 import java.util.List;
7 import java.util.Optional;
8
9 /**
10  * Output port for Product persistence.
11  * <p>
12  * McCall Factor: Reusability -- decouples domain from infrastructure.
13  */
14 public interface ProductRepositoryPort {
15
16     Product save(Product product);
17
18     Optional<Product> findById(Long id);
19
20     List<Product> findAll();
21
22     List<Product> findById(Long categoryId);
23
24     List<Product> findByStatus(ProductStatus status);
25
26     List<Product> findByNameContainingIgnoreCase(String name);
27
28     boolean existsById(Long id);
29
30     void deleteById(Long id);
31 }
```

backend/src/main/java/com/dongato/inventory/application/ports/out/StockMovementRepositoryPort.java

Código fuente

```
1 package com.dongato.inventory.application.ports.out;
2
3 import com.dongato.inventory.domain.model.StockMovement;
4
5 import java.util.List;
6
7 /**
8  * Output port for StockMovement persistence.
9  * <p>
10  * McCall Factor: Integrity -- provides the contract for audit trail storage.
11  */
12 public interface StockMovementRepositoryPort {
13 }
```

```
14     StockMovement save(StockMovement movement);
15
16     List<StockMovement> findByProductId(Long productId);
17
18     List<StockMovement> findAll();
19 }
```

backend/src/main/java/com/dongato/inventory/application/usecases/CategoryUseCase.java

```
Código fuente

1  package com.dongato.inventory.application.usecases;
2
3  import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
4  import com.dongato.inventory.domain.exception.BusinessRuleException;
5  import com.dongato.inventory.domain.exception.ResourceNotFoundException;
6  import com.dongato.inventory.domain.model.Category;
7  import lombok.RequiredArgsConstructor;
8  import lombok.extern.slf4j.Slf4j;
9  import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.time.LocalDateTime;
13 import java.util.List;
14
15 /**
16  * Use case for Category management.
17  * <p>
18  * McCall Factors: Correctness (business rules), Reusability (framework-agnostic logic).
19  */
20 @Service
21 @RequiredArgsConstructor
22 @Slf4j
23 @Transactional(readOnly = true)
24 public class CategoryUseCase {
25
26     private final CategoryRepositoryPort categoryRepository;
27
28     @Transactional
29     public Category create(Category category) {
30         category.validate();
31
32         if (categoryRepository.existsByName(category.getName())) {
33             throw new BusinessRuleException(
34                 String.format("Category with name '%s' already exists", category.getName()));
35         }
36
37         category.setCreatedAt(LocalDateTime.now());
38         log.info("Creating category: {}", category.getName());
39         return categoryRepository.save(category);
40     }
41
42     public Category findById(Long id) {
43         return categoryRepository.findById(id)
44             .orElseThrow(() -> new ResourceNotFoundException("Category", id));
45     }
46
47     public List<Category> findAll() {
48         return categoryRepository.findAll();
49     }
50
51     @Transactional
52     public Category update(Long id, Category updated) {
53         Category existing = findById(id);
54         updated.validate();
55
56         existing.setName(updated.getName());
57         existing.setDescription(updated.getDescription());
58
59         log.info("Updating category id={}: {}", id, existing.getName());
60         return categoryRepository.save(existing);
61     }
62 }
```

```
61     }
62
63     @Transactional
64     public void delete(Long id) {
65         if (!categoryRepository.existsById(id)) {
66             throw new ResourceNotFoundException("Category", id);
67         }
68         log.info("Deleting category id={}", id);
69         categoryRepository.deleteById(id);
70     }
71 }
```

backend/src/main/java/com/dongato/inventory/application/usecases/ProductUseCase.java

Código fuente

```
1  package com.dongato.inventory.application.usecases;
2
3  import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
4  import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
5  import com.dongato.inventory.domain.exception.ResourceNotFoundException;
6  import com.dongato.inventory.domain.model.Product;
7  import com.dongato.inventory.domain.model.ProductStatus;
8  import lombok.RequiredArgsConstructor;
9  import lombok.extern.slf4j.Slf4j;
10 import org.springframework.stereotype.Service;
11 import org.springframework.transaction.annotation.Transactional;
12
13 import java.time.LocalDateTime;
14 import java.util.List;
15
16 /**
17  * Use case for Product management.
18  * <p>
19  * McCall Factors: Correctness (validation), Reusability (port-based), Integrity (status tracking).
20  */
21 @Service
22 @RequiredArgsConstructor
23 @Slf4j
24 @Transactional(readOnly = true)
25 public class ProductUseCase {
26
27     private final ProductRepositoryPort productRepository;
28     private final CategoryRepositoryPort categoryRepository;
29
30     @Transactional
31     public Product create(Product product) {
32         product.validate();
33         validateCategoryExists(product.getCategoryId());
34
35         if (product.getStock() == null) {
36             product.setStock(0);
37         }
38         product.recalculateStatus();
39         product.setCreatedAt(LocalDateTime.now());
40
41         log.info("Creating product: {} (category={})", product.getName(), product.getCategoryId());
42         return productRepository.save(product);
43     }
44
45     public Product findById(Long id) {
46         return productRepository.findById(id)
47             .orElseThrow(() -> new ResourceNotFoundException("Product", id));
48     }
49
50     public List<Product> findAll() {
51         return productRepository.findAll();
52     }
53
54     public List<Product> findByCategoryId(Long categoryId) {
55         validateCategoryExists(categoryId);
```

```

56     return productRepository.findById(categoryId);
57 }
58
59 public List<Product> findByStatus(ProductStatus status) {
60     return productRepository.findByStatus(status);
61 }
62
63 public List<Product> searchByName(String name) {
64     return productRepository.findByNameContainingIgnoreCase(name);
65 }
66
67 @Transactional
68 public Product update(Long id, Product updated) {
69     Product existing = findById(id);
70     updated.validate();
71
72     if (updated.getCategoryId() != null) {
73         validateCategoryExists(updated.getCategoryId());
74         existing.setCategoryId(updated.getCategoryId());
75     }
76
77     existing.setName(updated.getName());
78     existing.setPrice(updated.getPrice());
79     // Stock is NOT updated directly here -- use StockMovement for traceability
80     existing.recalculateStatus();
81
82     log.info("Updating product id={}: {}", id, existing.getName());
83     return productRepository.save(existing);
84 }
85
86 @Transactional
87 public void delete(Long id) {
88     if (!productRepository.existsById(id)) {
89         throw new ResourceNotFoundException("Product", id);
90     }
91     log.info("Deleting product id={}", id);
92     productRepository.deleteById(id);
93 }
94
95 private void validateCategoryExists(Long categoryId) {
96     if (categoryId != null && !categoryRepository.existsById(categoryId)) {
97         throw new ResourceNotFoundException("Category", categoryId);
98     }
99 }
100 }

```

backend/src/main/java/com/dongato/inventory/application/usecases/StockMovementUseCase.java

Código fuente

```

1 package com.dongato.inventory.application.usecases;
2
3 import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
4 import com.dongato.inventory.application.ports.out.StockMovementRepositoryPort;
5 import com.dongato.inventory.domain.exception.InsufficientStockException;
6 import com.dongato.inventory.domain.exception.ResourceNotFoundException;
7 import com.dongato.inventory.domain.model.MovementType;
8 import com.dongato.inventory.domain.model.Product;
9 import com.dongato.inventory.domain.model.StockMovement;
10 import lombok.RequiredArgsConstructor;
11 import lombok.extern.slf4j.Slf4j;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.time.LocalDateTime;
16 import java.util.List;
17
18 /**
19  * Use case for StockMovement management -- the audit trail for all inventory changes.
20  * <p>
21  * Every stock modification MUST go through this use case to ensure traceability.

```

```

22  * McCall Factors: Integrity (audit trail), Correctness (stock consistency).
23  */
24  @Service
25  @RequiredArgsConstructor
26  @Slf4j
27  @Transactional(readOnly = true)
28  public class StockMovementUseCase {
29
30      private final StockMovementRepositoryPort movementRepository;
31      private final ProductRepositoryPort productRepository;
32
33      /**
34       * Registers a stock movement and updates the product stock accordingly.
35       * ENTRADA adds stock; SALIDA removes stock.
36       *
37       * @param movement the stock movement to register
38       * @return the persisted stock movement
39       */
40      @Transactional
41      public StockMovement registerMovement(StockMovement movement) {
42          movement.validate();
43
44          Product product = productRepository.findById(movement.getProductId())
45              .orElseThrow(() -> new ResourceNotFoundException("Product", movement.getProductId()));
46
47          if (movement.getType() == MovementType.ENTRADA) {
48              product.addStock(movement.getQuantity());
49              log.info("Stock ENTRY: product='{}', qty={}, reason={}",
50                  product.getName(), movement.getQuantity(), movement.getReason());
51          } else {
52              try {
53                  product.removeStock(movement.getQuantity());
54              } catch (IllegalStateException e) {
55                  throw new InsufficientStockException(
56                      product.getName(), product.getStock(), movement.getQuantity());
57              }
58              log.info("Stock EXIT: product='{}', qty={}, reason={}",
59                  product.getName(), movement.getQuantity(), movement.getReason());
60          }
61
62          productRepository.save(product);
63
64          movement.setCreatedAt(LocalDate.now());
65          return movementRepository.save(movement);
66      }
67
68      public List<StockMovement> findByProductId(Long productId) {
69          if (!productRepository.existsById(productId)) {
70              throw new ResourceNotFoundException("Product", productId);
71          }
72          return movementRepository.findByProductId(productId);
73      }
74
75      public List<StockMovement> findAll() {
76          return movementRepository.findAll();
77      }
78  }

```

A.5 Infraestructura: Persistencia

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/CategoryEntity.java

Código fuente

```

1  package com.dongato.inventory.infrastructure.persistence.entity;
2
3  import jakarta.persistence.*;
4  import lombok.AllArgsConstructor;
5  import lombok.Builder;

```

```

6  import lombok.Data;
7  import lombok.NoArgsConstructor;
8
9  import java.time.LocalDateTime;
10
11 /**
12  * JPA Entity for the 'category' table.
13  * McCall Factor: Portability -- ORM abstracts database specifics.
14  */
15 @Entity
16 @Table(name = "category")
17 @Data
18 @Builder
19 @AllArgsConstructor
20 @NoArgsConstructor
21 public class CategoryEntity {
22
23     @Id
24     @GeneratedValue(strategy = GenerationType.IDENTITY)
25     private Long id;
26
27     @Column(nullable = false, unique = true, length = 100)
28     private String name;
29
30     @Column(length = 255)
31     private String description;
32
33     @Column(name = "created_at", nullable = false, updatable = false)
34     private LocalDateTime createdAt;
35
36     @PrePersist
37     protected void onCreate() {
38         if (this.createdAt == null) {
39             this.createdAt = LocalDateTime.now();
40         }
41     }
42 }

```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/ProductEntity.java

Código fuente

```

1  package com.dongato.inventory.infrastructure.persistence.entity;
2
3  import com.dongato.inventory.domain.model.ProductStatus;
4  import jakarta.persistence.*;
5  import lombok.AllArgsConstructor;
6  import lombok.Builder;
7  import lombok.Data;
8  import lombok.NoArgsConstructor;
9
10 import java.math.BigDecimal;
11 import java.time.LocalDateTime;
12
13 /**
14  * JPA Entity for the 'product' table.
15  * McCall Factor: Portability -- ORM-managed persistence.
16  */
17 @Entity
18 @Table(name = "product")
19 @Data
20 @Builder
21 @AllArgsConstructor
22 @NoArgsConstructor
23 public class ProductEntity {
24
25     @Id
26     @GeneratedValue(strategy = GenerationType.IDENTITY)
27     private Long id;
28
29     @Column(name = "category_id", nullable = false)

```



```
30     private Long categoryId;
31
32     @Column(nullable = false, length = 150)
33     private String name;
34
35     @Column(nullable = false, precision = 10, scale = 2)
36     private BigDecimal price;
37
38     @Column(nullable = false)
39     private Integer stock;
40
41     @Enumerated(EnumType.STRING)
42     @Column(nullable = false, length = 20)
43     private ProductStatus status;
44
45     @Column(name = "created_at", nullable = false, updatable = false)
46     private LocalDateTime createdAt;
47
48     @ManyToOne(fetch = FetchType.LAZY)
49     @JoinColumn(name = "category_id", insertable = false, updatable = false)
50     private CategoryEntity category;
51
52     @PrePersist
53     protected void onCreate() {
54         if (this.createdAt == null) {
55             this.createdAt = LocalDateTime.now();
56         }
57     }
58 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/entity/StockMovementEntity.java

Código fuente

```
1 package com.dongato.inventory.infrastructure.persistence.entity;
2
3 import com.dongato.inventory.domain.model.MovementReason;
4 import com.dongato.inventory.domain.model.MovementType;
5 import jakarta.persistence.*;
6 import lombok.AllArgsConstructor;
7 import lombok.Builder;
8 import lombok.Data;
9 import lombok.NoArgsConstructor;
10
11 import java.time.LocalDateTime;
12
13 /**
14  * JPA Entity for the 'stock_movement' table.
15  * McCall Factor: Integrity -- persists audit trail for inventory changes.
16  */
17 @Entity
18 @Table(name = "stock_movement")
19 @Data
20 @Builder
21 @AllArgsConstructor
22 @NoArgsConstructor
23 public class StockMovementEntity {
24
25     @Id
26     @GeneratedValue(strategy = GenerationType.IDENTITY)
27     private Long id;
28
29     @Column(name = "product_id", nullable = false)
30     private Long productId;
31
32     @Enumerated(EnumType.STRING)
33     @Column(nullable = false, length = 10)
34     private MovementType type;
35
36     @Column(nullable = false)
37     private Integer quantity;
```

```
38
39 @Enumerated(EnumType.STRING)
40 @Column(nullable = false, length = 20)
41 private MovementReason reason;
42
43 @Column(name = "created_at", nullable = false, updatable = false)
44 private LocalDateTime createdAt;
45
46 @ManyToOne(fetch = FetchType.LAZY)
47 @JoinColumn(name = "product_id", insertable = false, updatable = false)
48 private ProductEntity product;
49
50 @PrePersist
51 protected void onCreate() {
52     if (this.createdAt == null) {
53         this.createdAt = LocalDateTime.now();
54     }
55 }
56 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/CategoryPersistenceMapper.java

Código fuente

```
1 package com.dongato.inventory.infrastructure.persistence.mapper;
2
3 import com.dongato.inventory.domain.model.Category;
4 import com.dongato.inventory.infrastructure.persistence.entity.CategoryEntity;
5
6 /**
7  * Maps between Category domain model and CategoryEntity JPA entity.
8  * <p>
9  * McCall Factor: Maintainability -- isolates domain from persistence representation.
10  */
11 public final class CategoryPersistenceMapper {
12
13     private CategoryPersistenceMapper() {
14         // Utility class -- prevent instantiation
15     }
16
17     public static Category toDomain(CategoryEntity entity) {
18         if (entity == null) return null;
19         return Category.builder()
20             .id(entity.getId())
21             .name(entity.getName())
22             .description(entity.getDescription())
23             .createdAt(entity.getCreatedAt())
24             .build();
25     }
26
27     public static CategoryEntity toEntity(Category domain) {
28         if (domain == null) return null;
29         return CategoryEntity.builder()
30             .id(domain.getId())
31             .name(domain.getName())
32             .description(domain.getDescription())
33             .createdAt(domain.getCreatedAt())
34             .build();
35     }
36 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/ProductPersistenceMapper.java

```
Código fuente

1 package com.dongato.inventory.infrastructure.persistence.mapper;
2
3 import com.dongato.inventory.domain.model.Product;
4 import com.dongato.inventory.infrastructure.persistence.entity.ProductEntity;
5
6 /**
7  * Maps between Product domain model and ProductEntity JPA entity.
8  * <p>
9  * McCall Factor: Maintainability -- decouples domain representation from ORM.
10 */
11 public final class ProductPersistenceMapper {
12
13     private ProductPersistenceMapper() {
14         // Utility class -- prevent instantiation
15     }
16
17     public static Product toDomain(ProductEntity entity) {
18         if (entity == null) return null;
19         return Product.builder()
20             .id(entity.getId())
21             .name(entity.getName())
22             .price(entity.getPrice())
23             .stock(entity.getStock())
24             .status(entity.getStatus())
25             .categoryId(entity.getCategoryId())
26             .createdAt(entity.getCreatedAt())
27             .build();
28     }
29
30     public static ProductEntity toEntity(Product domain) {
31         if (domain == null) return null;
32         return ProductEntity.builder()
33             .id(domain.getId())
34             .name(domain.getName())
35             .price(domain.getPrice())
36             .stock(domain.getStock())
37             .status(domain.getStatus())
38             .categoryId(domain.getCategoryId())
39             .createdAt(domain.getCreatedAt())
40             .build();
41     }
42 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/mapper/StockMovementPersistenceMapper.java

```
Código fuente

1 package com.dongato.inventory.infrastructure.persistence.mapper;
2
3 import com.dongato.inventory.domain.model.StockMovement;
4 import com.dongato.inventory.infrastructure.persistence.entity.StockMovementEntity;
5
6 /**
7  * Maps between StockMovement domain model and StockMovementEntity JPA entity.
8  * <p>
9  * McCall Factor: Maintainability -- isolates audit data from persistence details.
10 */
11 public final class StockMovementPersistenceMapper {
12
13     private StockMovementPersistenceMapper() {
14         // Utility class -- prevent instantiation
15     }
16
17     public static StockMovement toDomain(StockMovementEntity entity) {
18         if (entity == null) return null;
19         return StockMovement.builder()
```

```
20         id(entity.getId())
21         productId(entity.getProductId())
22         type(entity.getType())
23         quantity(entity.getQuantity())
24         reason(entity.getReason())
25         createdAt(entity.getCreatedAt())
26         build();
27     }
28
29     public static StockMovementEntity toEntity(StockMovement domain) {
30         if (domain == null) return null;
31         return StockMovementEntity.builder()
32             .id(domain.getId())
33             .productId(domain.getProductId())
34             .type(domain.getType())
35             .quantity(domain.getQuantity())
36             .reason(domain.getReason())
37             .createdAt(domain.getCreatedAt())
38             .build();
39     }
40 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaCategoryRepository.java

Código fuente

```
1 package com.dongato.inventory.infrastructure.persistence.repository;
2
3 import com.dongato.inventory.infrastructure.persistence.entity.CategoryEntity;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 /**
8  * Spring Data JPA repository for Category.
9  */
10 @Repository
11 public interface JpaCategoryRepository extends JpaRepository<CategoryEntity, Long> {
12
13     boolean existsByNameIgnoreCase(String name);
14 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaProductRepository.java

Código fuente

```
1 package com.dongato.inventory.infrastructure.persistence.repository;
2
3 import com.dongato.inventory.domain.model.ProductStatus;
4 import com.dongato.inventory.infrastructure.persistence.entity.ProductEntity;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7
8 import java.util.List;
9
10 /**
11  * Spring Data JPA repository for Product.
12  */
13 @Repository
14 public interface JpaProductRepository extends JpaRepository<ProductEntity, Long> {
15
16     List<ProductEntity> findByCategoryId(Long categoryId);
17
18     List<ProductEntity> findByStatus(ProductStatus status);
19
20     List<ProductEntity> findByNameContainingIgnoreCase(String name);
21 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/repository/JpaStockMovementRepository.java

```
Código fuente

1 package com.dongato.inventory.infrastructure.persistence.repository;
2
3 import com.dongato.inventory.infrastructure.persistence.entity.StockMovementEntity;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.List;
8
9 /**
10  * Spring Data JPA repository for StockMovement.
11  */
12 @Repository
13 public interface JpaStockMovementRepository extends JpaRepository<StockMovementEntity, Long> {
14
15     List<StockMovementEntity> findByProductIdOrderByCreatedAtDesc(Long productId);
16 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/CategoryRepositoryAdapter.java

```
Código fuente

1 package com.dongato.inventory.infrastructure.persistence.adapter;
2
3 import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
4 import com.dongato.inventory.domain.model.Category;
5 import com.dongato.inventory.infrastructure.persistence.mapper.CategoryPersistenceMapper;
6 import com.dongato.inventory.infrastructure.persistence.repository.JpaCategoryRepository;
7 import lombok.RequiredArgsConstructor;
8 import org.springframework.stereotype.Component;
9
10 import java.util.List;
11 import java.util.Optional;
12 import java.util.stream.Collectors;
13
14 /**
15  * Adapter that implements CategoryRepositoryPort using Spring Data JPA.
16  * <p>
17  * McCall Factor: Portability -- swappable persistence implementation.
18  */
19 @Component
20 @RequiredArgsConstructor
21 public class CategoryRepositoryAdapter implements CategoryRepositoryPort {
22
23     private final JpaCategoryRepository jpaRepository;
24
25     @Override
26     public Category save(Category category) {
27         var entity = CategoryPersistenceMapper.toEntity(category);
28         var saved = jpaRepository.save(entity);
29         return CategoryPersistenceMapper.toDomain(saved);
30     }
31
32     @Override
33     public Optional<Category> findById(Long id) {
34         return jpaRepository.findById(id)
35             .map(CategoryPersistenceMapper::toDomain);
36     }
37
38     @Override
39     public List<Category> findAll() {
40         return jpaRepository.findAll().stream()
41             .map(CategoryPersistenceMapper::toDomain)
42             .collect(Collectors.toList());
43     }
44
45     @Override
```

```
46     public boolean existsById(Long id) {
47         return jpaRepository.existsById(id);
48     }
49
50     @Override
51     public boolean existsByName(String name) {
52         return jpaRepository.existsByNameIgnoreCase(name);
53     }
54
55     @Override
56     public void deleteById(Long id) {
57         jpaRepository.deleteById(id);
58     }
59 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapter/ProductRepositoryAdapter.java

Código fuente

```
1  package com.dongato.inventory.infrastructure.persistence.adapter;
2
3  import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
4  import com.dongato.inventory.domain.model.Product;
5  import com.dongato.inventory.domain.model.ProductStatus;
6  import com.dongato.inventory.infrastructure.persistence.mapper.ProductPersistenceMapper;
7  import com.dongato.inventory.infrastructure.persistence.repository.JpaProductRepository;
8  import lombok.RequiredArgsConstructor;
9  import org.springframework.stereotype.Component;
10
11  import java.util.List;
12  import java.util.Optional;
13  import java.util.stream.Collectors;
14
15  /**
16   * Adapter that implements ProductRepositoryPort using Spring Data JPA.
17   * <p>
18   * McCall Factor: Portability -- database-agnostic through JPA abstraction.
19   */
20  @Component
21  @RequiredArgsConstructor
22  public class ProductRepositoryAdapter implements ProductRepositoryPort {
23
24      private final JpaProductRepository jpaRepository;
25
26      @Override
27      public Product save(Product product) {
28          var entity = ProductPersistenceMapper.toEntity(product);
29          var saved = jpaRepository.save(entity);
30          return ProductPersistenceMapper.toDomain(saved);
31      }
32
33      @Override
34      public Optional<Product> findById(Long id) {
35          return jpaRepository.findById(id)
36              .map(ProductPersistenceMapper::toDomain);
37      }
38
39      @Override
40      public List<Product> findAll() {
41          return jpaRepository.findAll().stream()
42              .map(ProductPersistenceMapper::toDomain)
43              .collect(Collectors.toList());
44      }
45
46      @Override
47      public List<Product> findByIdByCategoryId(Long categoryId) {
48          return jpaRepository.findByIdByCategoryId(categoryId).stream()
49              .map(ProductPersistenceMapper::toDomain)
50              .collect(Collectors.toList());
51      }
52 }
```

```
53     @Override
54     public List<Product> findByStatus(ProductStatus status) {
55         return jpaRepository.findByStatus(status).stream()
56             .map(ProductPersistenceMapper::toDomain)
57             .collect(Collectors.toList());
58     }
59
60     @Override
61     public List<Product> findByNameContainingIgnoreCase(String name) {
62         return jpaRepository.findByNameContainingIgnoreCase(name).stream()
63             .map(ProductPersistenceMapper::toDomain)
64             .collect(Collectors.toList());
65     }
66
67     @Override
68     public boolean existsById(Long id) {
69         return jpaRepository.existsById(id);
70     }
71
72     @Override
73     public void deleteById(Long id) {
74         jpaRepository.deleteById(id);
75     }
76 }
```

backend/src/main/java/com/dongato/inventory/infrastructure/persistence/adapters/StockMovementRepositoryAdapter.java

Código fuente

```
1  package com.dongato.inventory.infrastructure.persistence.adapters;
2
3  import com.dongato.inventory.application.ports.out.StockMovementRepositoryPort;
4  import com.dongato.inventory.domain.model.StockMovement;
5  import com.dongato.inventory.infrastructure.persistence.mapper.StockMovementPersistenceMapper;
6  import com.dongato.inventory.infrastructure.persistence.repository.JpaStockMovementRepository;
7  import lombok.RequiredArgsConstructor;
8  import org.springframework.stereotype.Component;
9
10 import java.util.List;
11 import java.util.stream.Collectors;
12
13 /**
14  * Adapter that implements StockMovementRepositoryPort using Spring Data JPA.
15  * <p>
16  * McCall Factor: Integrity -- ensures audit data is persisted reliably.
17  */
18 @Component
19 @RequiredArgsConstructor
20 public class StockMovementRepositoryAdapter implements StockMovementRepositoryPort {
21
22     private final JpaStockMovementRepository jpaRepository;
23
24     @Override
25     public StockMovement save(StockMovement movement) {
26         var entity = StockMovementPersistenceMapper.toEntity(movement);
27         var saved = jpaRepository.save(entity);
28         return StockMovementPersistenceMapper.toDomain(saved);
29     }
30
31     @Override
32     public List<StockMovement> findByProductId(Long productId) {
33         return jpaRepository.findByProductIdOrderByCreatedAtDesc(productId).stream()
34             .map(StockMovementPersistenceMapper::toDomain)
35             .collect(Collectors.toList());
36     }
37
38     @Override
39     public List<StockMovement> findAll() {
40         return jpaRepository.findAll().stream()
41             .map(StockMovementPersistenceMapper::toDomain)
42             .collect(Collectors.toList());
43     }
44 }
```

```
43     }  
44 }
```

A.6 Infraestructura: Seguridad y Configuración

backend/src/main/java/com/dongato/inventory/infrastructure/config/DataSeeder.java

Código fuente

```
1 package com.dongato.inventory.infrastructure.config;  
2  
3 import com.dongato.inventory.infrastructure.persistence.entity.CategoryEntity;  
4 import com.dongato.inventory.infrastructure.persistence.entity.ProductEntity;  
5 import com.dongato.inventory.infrastructure.persistence.repository.JpaCategoryRepository;  
6 import com.dongato.inventory.infrastructure.persistence.repository.JpaProductRepository;  
7 import com.dongato.inventory.domain.model.ProductStatus;  
8 import lombok.RequiredArgsConstructor;  
9 import lombok.extern.slf4j.Slf4j;  
10 import org.springframework.boot.CommandLineRunner;  
11 import org.springframework.context.annotation.Bean;  
12 import org.springframework.context.annotation.Configuration;  
13  
14 import java.math.BigDecimal;  
15 import java.time.LocalDateTime;  
16  
17 /**  
18  * Seeds the database with initial cafeteria data on first run.  
19  * Only inserts if the database is empty.  
20  */  
21 @Configuration  
22 @RequiredArgsConstructor  
23 @Slf4j  
24 public class DataSeeder {  
25  
26     @Bean  
27     public CommandLineRunner seedData(  
28         JpaCategoryRepository categoryRepo,  
29         JpaProductRepository productRepo) {  
30         return args -> {  
31             if (categoryRepo.count() > 0) {  
32                 log.info("Database already seeded -- skipping");  
33                 return;  
34             }  
35  
36             log.info("Seeding initial data for Cafetería Don Gato...");  
37             LocalDateTime now = LocalDateTime.now();  
38  
39             // Categories  
40             CategoryEntity cafe = categoryRepo.save(CategoryEntity.builder()  
41                 .name("CAFE").description("Bebidas de café preparadas").createdAt(now).build());  
42             CategoryEntity bebida = categoryRepo.save(CategoryEntity.builder()  
43                 .name("BEBIDA").description("Bebidas frías y calientes no-café").createdAt(now).build());  
44  
45             ;  
46             CategoryEntity snack = categoryRepo.save(CategoryEntity.builder()  
47                 .name("SNACK").description("Bocadillos y acompañamientos").createdAt(now).build());  
48  
49             // Products -- CAFE  
50             productRepo.save(ProductEntity.builder()  
51                 .name("Americano").price(new BigDecimal("2.50")).stock(50)  
52                 .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());  
53             productRepo.save(ProductEntity.builder()  
54                 .name("Cappuccino").price(new BigDecimal("3.50")).stock(40)  
55                 .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());  
56             productRepo.save(ProductEntity.builder()  
57                 .name("Latte").price(new BigDecimal("3.75")).stock(35)  
58                 .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());  
59             productRepo.save(ProductEntity.builder()  
60                 .name("Espresso").price(new BigDecimal("2.00")).stock(60)  
61                 .status(ProductStatus.AVAILABLE).categoryId(cafe.getId()).createdAt(now).build());  
62         }  
63     }  
64 }
```



```

60     productRepo save ProductEntity builder()
61         name("Mocha") price(new BigDecimal("4.00")).stock(30)
62         status ProductStatus AVAILABLE).categoryId(caffe.getId()).createdAt(now).build();
63
64     // Products -- BEBIDA
65     productRepo save ProductEntity builder()
66         name("Jugo de Naranja") price(new BigDecimal("2.50")).stock(25)
67         status ProductStatus AVAILABLE).categoryId(bebida.getId()).createdAt(now).build();
68     productRepo save ProductEntity builder()
69         name("Té Verde") price(new BigDecimal("2.00")).stock(40)
70         status ProductStatus AVAILABLE).categoryId(bebida.getId()).createdAt(now).build();
71     productRepo save ProductEntity builder()
72         name("Agua Mineral") price(new BigDecimal("1.50")).stock(100)
73         status ProductStatus AVAILABLE).categoryId(bebida.getId()).createdAt(now).build();
74
75     // Products -- SNACK
76     productRepo save ProductEntity builder()
77         name("Croissant") price(new BigDecimal("2.75")).stock(20)
78         status ProductStatus AVAILABLE).categoryId(snack.getId()).createdAt(now).build();
79     productRepo save ProductEntity builder()
80         name("Muffin de Arándano") price(new BigDecimal("3.00")).stock(15)
81         status ProductStatus AVAILABLE).categoryId(snack.getId()).createdAt(now).build();
82     productRepo save ProductEntity builder()
83         name("Sándwich Club") price(new BigDecimal("5.50")).stock(10)
84         status ProductStatus AVAILABLE).categoryId(snack.getId()).createdAt(now).build();
85
86     log.info("Database seeded with {} categories and {} products",
87             categoryRepo.count(), productRepo.count());
88 }
89 }
90 }

```

backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtAuthenticationFilter.java

Código fuente

```

1  package com.dongato.inventory.infrastructure.security;
2
3  import jakarta.servlet.FilterChain;
4  import jakarta.servlet.ServletException;
5  import jakarta.servlet.http.HttpServletRequest;
6  import jakarta.servlet.http.HttpServletResponse;
7  import lombok.extern.slf4j.Slf4j;
8  import org.springframework.context.annotation.Lazy;
9  import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
10 import org.springframework.security.core.context.SecurityContextHolder;
11 import org.springframework.security.core.userdetails.UserDetails;
12 import org.springframework.security.core.userdetails.UserDetailsService;
13 import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
14 import org.springframework.stereotype.Component;
15 import org.springframework.util.StringUtils;
16 import org.springframework.web.filter.OncePerRequestFilter;
17
18 import java.io.IOException;
19
20 /**
21  * JWT Authentication Filter -- intercepts every request to validate the Bearer token.
22  * <p>
23  * McCall Factor: Integrity -- enforces authentication on every request.
24  */
25 @Component
26 @Slf4j
27 public class JwtAuthenticationFilter extends OncePerRequestFilter {
28
29     private final JwtTokenProvider jwtTokenProvider;
30     private final UserDetailsService userDetailsService;
31
32     public JwtAuthenticationFilter(
33         JwtTokenProvider jwtTokenProvider,
34         @Lazy UserDetailsService userDetailsService) {
35         this.jwtTokenProvider = jwtTokenProvider;

```

```

36     this userDetailsService = userDetailsService;
37 }
38
39 @Override
40 protected void doFilterInternal(
41     HttpServletRequest request,
42     HttpServletResponse response,
43     FilterChain filterChain) throws ServletException, IOException {
44
45     String token = extractTokenFromRequest(request);
46
47     if (StringUtils.hasText(token) && jwtTokenProvider.validateToken(token)) {
48         String username = jwtTokenProvider.getUsernameFromToken(token);
49         UserDetails userDetails = userDetailsService.loadUserByUsername(username);
50
51         UsernamePasswordAuthenticationToken authToken =
52             new UsernamePasswordAuthenticationToken(
53                 userDetails, null, userDetails.getAuthorities());
54         authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
55
56         SecurityContextHolder.getContext().setAuthentication(authToken);
57         log.debug("Authenticated user: {}", username);
58     }
59
60     filterChain.doFilter(request, response);
61 }
62
63 private String extractTokenFromRequest(HttpServletRequest request) {
64     String bearerToken = request.getHeader("Authorization");
65     if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
66         return bearerToken.substring(7);
67     }
68     return null;
69 }
70 }

```

backend/src/main/java/com/dongato/inventory/infrastructure/security/JwtTokenProvider.java

Código fuente

```

1  package com.dongato.inventory.infrastructure.security;
2
3  import io.jsonwebtoken.*;
4  import io.jsonwebtoken.security.Keys;
5  import org.springframework.beans.factory.annotation.Value;
6  import org.springframework.stereotype.Component;
7
8  import javax.crypto.SecretKey;
9  import java.nio.charset.StandardCharsets;
10 import java.util.Date;
11
12 /**
13  * JWT utility for token generation and validation.
14  * <p>
15  * McCall Factor: Integrity -- secure authentication tokens.
16  * Uses HMAC-SHA256 signing with configurable expiration.
17  */
18 @Component
19 public class JwtTokenProvider {
20
21     private final SecretKey secretKey;
22     private final long expirationMs;
23
24     public JwtTokenProvider(
25         @Value("${jwt.secret:CafeteriaDonGatoSecretKeyForJWTToken2026MustBeAtLeast256Bits}") String
26         secret,
27         @Value("${jwt.expiration-ms:3600000}") long expirationMs) {
28         this.secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
29         this.expirationMs = expirationMs;
30     }

```

```
31  /**
32   * Generates a JWT token for the given username.
33   */
34  public String generateToken(String username) {
35      Date now = new Date();
36      Date expiry = new Date(now.getTime() + expirationMs);
37
38      return Jwts.builder()
39          .subject(username)
40          .issuedAt(now)
41          .expiration(expiry)
42          .signWith(secretKey)
43          .compact();
44  }
45
46  /**
47   * Extracts the username from a JWT token.
48   */
49  public String getUsernameFromToken(String token) {
50      return Jwts.parser()
51          .verifyWith(secretKey)
52          .build()
53          .parseSignedClaims(token)
54          .getPayload()
55          .getSubject();
56  }
57
58  /**
59   * Validates the JWT token.
60   */
61  public boolean validateToken(String token) {
62      try {
63          Jwts.parser()
64              .verifyWith(secretKey)
65              .build()
66              .parseSignedClaims(token);
67          return true;
68      } catch (JwtException | IllegalArgumentException e) {
69          return false;
70      }
71  }
72
73  public long getExpirationMs() {
74      return expirationMs;
75  }
76  }
```

backend/src/main/java/com/dongato/inventory/infrastructure/security/SecurityConfig.java

Código fuente

```
1  package com.dongato.inventory.infrastructure.security;
2
3  import lombok.RequiredArgsConstructor;
4  import org.springframework.context.annotation.Bean;
5  import org.springframework.context.annotation.Configuration;
6  import org.springframework.http.HttpMethod;
7  import org.springframework.security.authentication.AuthenticationManager;
8  import org.springframework.security.config.annotation.authentication.configuration.
9      AuthenticationConfiguration;
10 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
11 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
12 import org.springframework.security.config.http.SessionCreationPolicy;
13 import org.springframework.security.core.userdetails.User;
14 import org.springframework.security.core.userdetails.UserDetailsService;
15 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
16 import org.springframework.security.crypto.password.PasswordEncoder;
17 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
18 import org.springframework.security.web.SecurityFilterChain;
19 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
20 import org.springframework.web.cors.CorsConfiguration;
```

```

20 import org.springframework.web.cors.CorsConfigurationSource;
21 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
22
23 import java.util.List;
24
25 /**
26  * Spring Security configuration with JWT-based stateless authentication.
27  * <p>
28  * McCall Factor: Integrity -- enforces authentication and authorization.
29  * Security hardening: CSRF disabled (stateless), CORS configured, session-less.
30  */
31 @Configuration
32 @EnableWebSecurity
33 @RequiredArgsConstructor
34 public class SecurityConfig {
35
36     private final JwtAuthenticationFilter jwtAuthenticationFilter;
37
38     @Bean
39     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
40         http
41             .csrf(csrf -> csrf.disable()) // Stateless API -- CSRF not needed
42             .cors(cors -> cors.configurationSource(corsConfigurationSource()))
43             .sessionManagement(session ->
44                 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
45             .authorizeHttpRequests(auth -> auth
46                 // Public endpoints
47                 .requestMatchers("/api/v1/auth/**").permitAll()
48                 .requestMatchers(HttpMethod.GET, "/api/v1/categories/**").permitAll()
49                 .requestMatchers(HttpMethod.GET, "/api/v1/products/**").permitAll()
50                 // All write operations require authentication
51                 .anyRequest().authenticated()
52             )
53             .addFilterBefore(jwtAuthenticationFilter
54                 UsernamePasswordAuthenticationFilter.class);
55
56         return http.build();
57     }
58
59     @Bean
60     public UserDetailsService userDetailsService>PasswordEncoder passwordEncoder) {
61         // In-memory user for educational project -- production would use a DB-backed UserDetailsService
62         var admin = User.builder()
63             .username("admin")
64             .password(passwordEncoder.encode("dongato2026"))
65             .roles("ADMIN")
66             .build();
67
68         var user = User.builder()
69             .username("cajero")
70             .password(passwordEncoder.encode("cafe123"))
71             .roles("USER")
72             .build();
73
74         return new InMemoryUserDetailsManager(admin, user);
75     }
76
77     @Bean
78     public PasswordEncoder passwordEncoder() {
79         return new BCryptPasswordEncoder();
80     }
81
82     @Bean
83     public AuthenticationManager authenticationManager(
84         AuthenticationConfiguration authConfig) throws Exception {
85         return authConfig.getAuthenticationManager();
86     }
87
88     @Bean
89     public CorsConfigurationSource corsConfigurationSource() {
90         CorsConfiguration config = new CorsConfiguration();
91         config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:5173"));
92         config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
93         config.setAllowedHeaders(List.of("*"));
94         config.setAllowCredentials(true);

```

```
95         config.setMaxAge(3600L);
96
97         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
98         source.registerCorsConfiguration("/**", config);
99         return source;
100     }
101 }
```

A.7 Interfaces REST: Controladores

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/AuthController.java

```
Código fuente

1  package com.dongato.inventory.interfaces.rest.controller;
2
3  import com.dongato.inventory.infrastructure.security.JwtTokenProvider;
4  import com.dongato.inventory.interfaces.rest.dto.AuthRequestDTO;
5  import com.dongato.inventory.interfaces.rest.dto.AuthResponseDTO;
6  import jakarta.validation.Valid;
7  import lombok.RequiredArgsConstructor;
8  import lombok.extern.slf4j.Slf4j;
9  import org.springframework.http.ResponseEntity;
10 import org.springframework.security.authentication.AuthenticationManager;
11 import org.springframework.security.authentication.BadCredentialsException;
12 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
13 import org.springframework.security.core.Authentication;
14 import org.springframework.web.bind.annotation.PostMapping;
15 import org.springframework.web.bind.annotation.RequestBody;
16 import org.springframework.web.bind.annotation.RequestMapping;
17 import org.springframework.web.bind.annotation.RestController;
18
19 /**
20  * Authentication controller for JWT token generation.
21  * <p>
22  * McCall Factor: Integrity -- secure authentication endpoint.
23  */
24 @RestController
25 @RequestMapping("/api/v1/auth")
26 @RequiredArgsConstructor
27 @Slf4j
28 public class AuthController {
29
30     private final AuthenticationManager authenticationManager;
31     private final JwtTokenProvider jwtTokenProvider;
32
33     @PostMapping("/login")
34     public ResponseEntity<AuthResponseDTO> login(@Valid @RequestBody AuthRequestDTO request) {
35         try {
36             Authentication authentication = authenticationManager.authenticate(
37                 new UsernamePasswordAuthenticationToken(
38                     request.getUsername(), request.getPassword()));
39
40             String token = jwtTokenProvider.generateToken(authentication.getName());
41
42             AuthResponseDTO response = AuthResponseDTO.builder()
43                 .token(token)
44                 .type("Bearer")
45                 .username(authentication.getName())
46                 .expiresIn(jwtTokenProvider.getExpirationMs() / 1000)
47                 .build();
48
49             log.info("User '{}' authenticated successfully", authentication.getName());
50             return ResponseEntity.ok(response);
51
52         } catch (BadCredentialsException e) {
53             log.warn("Failed authentication attempt for user: {}", request.getUsername());
54             throw e;
55         }
56     }
57 }
```

```
56     }  
57 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/CategoryController.java

Código fuente

```
1 package com.dongato.inventory.interfaces.rest.controller;  
2  
3 import com.dongato.inventory.application.usecases.CategoryUseCase;  
4 import com.dongato.inventory.domain.model.Category;  
5 import com.dongato.inventory.interfaces.rest.dto.CategoryCreateDTO;  
6 import com.dongato.inventory.interfaces.rest.dto.CategoryResponseDTO;  
7 import com.dongato.inventory.interfaces.rest.mapper.CategoryRestMapper;  
8 import jakarta.validation.Valid;  
9 import lombok.RequiredArgsConstructor;  
10 import org.springframework.http.HttpStatus;  
11 import org.springframework.http.ResponseEntity;  
12 import org.springframework.web.bind.annotation.*;  
13  
14 import java.util.List;  
15 import java.util.stream.Collectors;  
16  
17 /**  
18  * REST Controller for Category management.  
19  * <p>  
20  * McCall Factors: Interoperability (REST standard), Integrity (input validation).  
21  */  
22 @RestController  
23 @RequestMapping("/api/v1/categories")  
24 @RequiredArgsConstructor  
25 public class CategoryController {  
26  
27     private final CategoryUseCase categoryUseCase;  
28  
29     @PostMapping  
30     public ResponseEntity<CategoryResponseDTO> create(@Valid @RequestBody CategoryCreateDTO dto) {  
31         Category category = CategoryRestMapper.toDomain(dto);  
32         Category created = categoryUseCase.create(category);  
33         return ResponseEntity.status(HttpStatus.CREATED)  
34             .body(CategoryRestMapper.toResponseDto(created));  
35     }  
36  
37     @GetMapping  
38     public ResponseEntity<List<CategoryResponseDTO>> findAll() {  
39         List<CategoryResponseDTO> categories = categoryUseCase.findAll().stream()  
40             .map(CategoryRestMapper::toResponseDto)  
41             .collect(Collectors.toList());  
42         return ResponseEntity.ok(categories);  
43     }  
44  
45     @GetMapping("/{id}")  
46     public ResponseEntity<CategoryResponseDTO> findById(@PathVariable Long id) {  
47         Category category = categoryUseCase.findById(id);  
48         return ResponseEntity.ok(CategoryRestMapper.toResponseDto(category));  
49     }  
50  
51     @PutMapping("/{id}")  
52     public ResponseEntity<CategoryResponseDTO> update(  
53         @PathVariable Long id,  
54         @Valid @RequestBody CategoryCreateDTO dto) {  
55         Category category = CategoryRestMapper.toDomain(dto);  
56         Category updated = categoryUseCase.update(id, category);  
57         return ResponseEntity.ok(CategoryRestMapper.toResponseDto(updated));  
58     }  
59  
60     @DeleteMapping("/{id}")  
61     public ResponseEntity<Void> delete(@PathVariable Long id) {  
62         categoryUseCase.delete(id);  
63         return ResponseEntity.noContent().build();  
64     }  
65 }
```

65 }

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/ProductController.java

Código fuente

```
1 package com.dongato.inventory.interfaces.rest.controller;
2
3 import com.dongato.inventory.application.usecases.ProductUseCase;
4 import com.dongato.inventory.domain.model.Product;
5 import com.dongato.inventory.domain.model.ProductStatus;
6 import com.dongato.inventory.interfaces.rest.dto.ProductCreateDTO;
7 import com.dongato.inventory.interfaces.rest.dto.ProductResponseDTO;
8 import com.dongato.inventory.interfaces.rest.dto.ProductUpdateDTO;
9 import com.dongato.inventory.interfaces.rest.mapper.ProductRestMapper;
10 import jakarta.validation.Valid;
11 import lombok.RequiredArgsConstructor;
12 import org.springframework.http.HttpStatus;
13 import org.springframework.http.ResponseEntity;
14 import org.springframework.web.bind.annotation.*;
15
16 import java.util.List;
17 import java.util.stream.Collectors;
18
19 /**
20  * REST Controller for Product management.
21  * <p>
22  * McCall Factors: Interoperability (REST standard), Correctness (validated input).
23  */
24 @RestController
25 @RequestMapping("/api/v1/products")
26 @RequiredArgsConstructor
27 public class ProductController {
28
29     private final ProductUseCase productUseCase;
30
31     @PostMapping
32     public ResponseEntity<ProductResponseDTO> create(@Valid @RequestBody ProductCreateDTO dto) {
33         Product product = ProductRestMapper.toDomain(dto);
34         Product created = productUseCase.create(product);
35         return ResponseEntity.status(HttpStatus.CREATED)
36             .body(ProductRestMapper.toResponseDto(created));
37     }
38
39     @GetMapping
40     public ResponseEntity<List<ProductResponseDTO>> findAll() {
41         List<ProductResponseDTO> products = productUseCase.findAll().stream()
42             .map(ProductRestMapper::toResponseDto)
43             .collect(Collectors.toList());
44         return ResponseEntity.ok(products);
45     }
46
47     @GetMapping("/{id}")
48     public ResponseEntity<ProductResponseDTO> findById(@PathVariable Long id) {
49         Product product = productUseCase.findById(id);
50         return ResponseEntity.ok(ProductRestMapper.toResponseDto(product));
51     }
52
53     @GetMapping("/category/{categoryId}")
54     public ResponseEntity<List<ProductResponseDTO>> findByCategory(@PathVariable Long categoryId) {
55         List<ProductResponseDTO> products = productUseCase.findByCategoryId(categoryId).stream()
56             .map(ProductRestMapper::toResponseDto)
57             .collect(Collectors.toList());
58         return ResponseEntity.ok(products);
59     }
60
61     @GetMapping("/status/{status}")
62     public ResponseEntity<List<ProductResponseDTO>> findByStatus(@PathVariable ProductStatus status) {
63         List<ProductResponseDTO> products = productUseCase.findByStatus(status).stream()
64             .map(ProductRestMapper::toResponseDto)
65             .collect(Collectors.toList());
```



```

66     return ResponseEntity.ok(products);
67 }
68
69 @GetMapping("/search")
70 public ResponseEntity<List<ProductResponseDTO>> searchByName(@RequestParam String name) {
71     List<ProductResponseDTO> products = productUseCase.searchByName(name).stream()
72         .map(ProductRestMapper::toResponseDto)
73         .collect(Collectors.toList());
74     return ResponseEntity.ok(products);
75 }
76
77 @PutMapping("/{id}")
78 public ResponseEntity<ProductResponseDTO> update(
79     @PathVariable Long id,
80     @Valid @RequestBody ProductUpdateDTO dto) {
81     Product product = ProductRestMapper.toDomain(dto);
82     Product updated = productUseCase.update(id, product);
83     return ResponseEntity.ok(ProductRestMapper.toResponseDto(updated));
84 }
85
86 @DeleteMapping("/{id}")
87 public ResponseEntity<Void> delete(@PathVariable Long id) {
88     productUseCase.delete(id);
89     return ResponseEntity.noContent().build();
90 }
91 }

```

backend/src/main/java/com/dongato/inventory/interfaces/rest/controller/StockMovementController.java

Código fuente

```

1  package com.dongato.inventory.interfaces.rest.controller;
2
3  import com.dongato.inventory.application.usecases.StockMovementUseCase;
4  import com.dongato.inventory.domain.model.StockMovement;
5  import com.dongato.inventory.interfaces.rest.dto.StockMovementCreateDTO;
6  import com.dongato.inventory.interfaces.rest.dto.StockMovementResponseDTO;
7  import com.dongato.inventory.interfaces.rest.mapper.StockMovementRestMapper;
8  import jakarta.validation.Valid;
9  import lombok.RequiredArgsConstructor;
10 import org.springframework.http.HttpStatus;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.web.bind.annotation.*;
13
14 import java.util.List;
15 import java.util.stream.Collectors;
16
17 /**
18  * REST Controller for Stock Movement management.
19  * <p>
20  * This is the ONLY way to modify product stock -- ensuring full audit trail.
21  * McCall Factors: Integrity (traceability), Interoperability (REST standard).
22  */
23 @RestController
24 @RequestMapping("/api/v1/stock-movements")
25 @RequiredArgsConstructor
26 public class StockMovementController {
27
28     private final StockMovementUseCase stockMovementUseCase;
29
30     @PostMapping
31     public ResponseEntity<StockMovementResponseDTO> register(
32         @Valid @RequestBody StockMovementCreateDTO dto) {
33         StockMovement movement = StockMovementRestMapper.toDomain(dto);
34         StockMovement registered = stockMovementUseCase.registerMovement(movement);
35         return ResponseEntity.status(HttpStatus.CREATED)
36             .body(StockMovementRestMapper.toResponseDto(registered));
37     }
38
39     @GetMapping
40     public ResponseEntity<List<StockMovementResponseDTO>> findAll() {

```



```

41     List<StockMovementResponseDTO> movements = stockMovementUseCase.findAll().stream()
42         .map(StockMovementRestMapper::toResponseDto)
43         .collect(Collectors.toList());
44     return ResponseEntity.ok(movements);
45 }
46
47 @GetMapping("/product/{productId}")
48 public ResponseEntity<List<StockMovementResponseDTO>> findByProduct(
49     @PathVariable Long productId) {
50     List<StockMovementResponseDTO> movements = stockMovementUseCase.findByProductId(productId)
51         .stream()
52         .map(StockMovementRestMapper::toResponseDto)
53         .collect(Collectors.toList());
54     return ResponseEntity.ok(movements);
55 }
56 }

```

A.8 Interfaces REST: DTOs, Errores y Mappers

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ApiErrorDTO.java

Código fuente

```

1  package com.dongato.inventory.interfaces.rest.dto;
2
3  import com.fasterxml.jackson.annotation.JsonInclude;
4  import lombok.AllArgsConstructor;
5  import lombok.Builder;
6  import lombok.Data;
7  import lombok.NoArgsConstructor;
8
9  import java.time.LocalDateTime;
10 import java.util.List;
11
12 /**
13  * Standardized API error response.
14  * McCall Factor: Usability -- consistent, informative error messages.
15  */
16 @Data
17 @Builder
18 @AllArgsConstructor
19 @NoArgsConstructor
20 @JsonInclude(JsonInclude.Include.NON_NULL)
21 public class ApiErrorDTO {
22
23     private int status;
24     private String error;
25     private String message;
26     private String path;
27     private LocalDateTime timestamp;
28     private List<FieldErrorDTO> fieldErrors;
29
30     @Data
31     @Builder
32     @AllArgsConstructor
33     @NoArgsConstructor
34     public static class FieldErrorDTO {
35         private String field;
36         private String message;
37     }
38 }

```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthRequestDTO.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import lombok.AllArgsConstructor;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7
8 /**
9  * DTO for authentication requests.
10  * McCall Factor: Integrity -- validates credentials at boundary.
11  */
12 @Data
13 @AllArgsConstructor
14 @NoArgsConstructor
15 public class AuthRequestDTO {
16
17     @NotBlank(message = "Username is required")
18     private String username;
19
20     @NotBlank(message = "Password is required")
21     private String password;
22 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/AuthResponseDTO.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Builder;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7
8 /**
9  * DTO for authentication responses (JWT token).
10  * McCall Factor: Integrity -- secure token delivery.
11  */
12 @Data
13 @Builder
14 @AllArgsConstructor
15 @NoArgsConstructor
16 public class AuthResponseDTO {
17
18     private String token;
19     private String type;
20     private String username;
21     private long expiresIn;
22 }
```

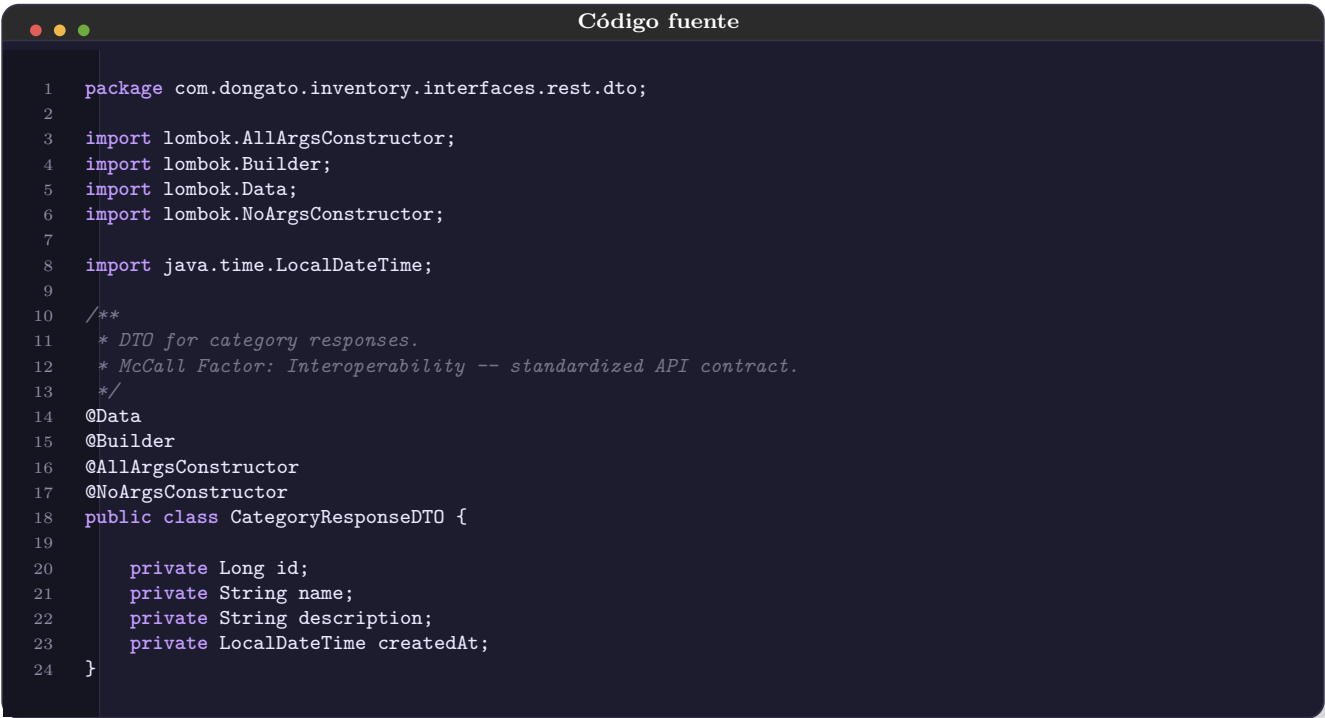
backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryCreatedDTO.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import jakarta.validation.constraints.NotBlank;
4 import jakarta.validation.constraints.Size;
5 import lombok.AllArgsConstructor;
6 import lombok.Builder;
7 import lombok.Data;
```

```
8 import lombok.NoArgsConstructor;
9
10 /**
11  * DTO for creating a category.
12  * McCall Factor: Correctness -- input validation at the boundary.
13  */
14 @Data
15 @Builder
16 @AllArgsConstructor
17 @NoArgsConstructor
18 public class CategoryCreateDTO {
19
20     @NotBlank(message = "Category name is required")
21     @Size(max = 100, message = "Category name must not exceed 100 characters")
22     private String name;
23
24     @Size(max = 255, message = "Description must not exceed 255 characters")
25     private String description;
26 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/CategoryResponseDTO.java



```
1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Builder;
5 import lombok.Data;
6 import lombok.NoArgsConstructor;
7
8 import java.time.LocalDateTime;
9
10 /**
11  * DTO for category responses.
12  * McCall Factor: Interoperability -- standardized API contract.
13  */
14 @Data
15 @Builder
16 @AllArgsConstructor
17 @NoArgsConstructor
18 public class CategoryResponseDTO {
19
20     private Long id;
21     private String name;
22     private String description;
23     private LocalDateTime createdAt;
24 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductCreateDTO.java



```
1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import jakarta.validation.constraints.*;
4 import lombok.AllArgsConstructor;
5 import lombok.Builder;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 import java.math.BigDecimal;
10
11 /**
12  * DTO for creating a product.
13  * McCall Factor: Correctness -- strict input validation.
14  */
```

```
15 @Data
16 @Builder
17 @AllArgsConstructor
18 @NoArgsConstructor
19 public class ProductCreatedTO {
20
21     @NotBlank(message = "Product name is required")
22     @Size max = 150, message = "Product name must not exceed 150 characters")
23     private String name;
24
25     @NotNull(message = "Price is required")
26     @DecimalMin(value = "0.00", message = "Price must be non-negative")
27     private BigDecimal price;
28
29     @Min(value = 0, message = "Initial stock must be non-negative")
30     private Integer stock;
31
32     @NotNull(message = "Category ID is required")
33     private Long categoryId;
34 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductResponseDTO.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import com.dongato.inventory.domain.model.ProductStatus;
4 import lombok.AllArgsConstructor;
5 import lombok.Builder;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 import java.math.BigDecimal;
10 import java.time.LocalDateTime;
11
12 /**
13  * DTO for product responses.
14  * McCall Factor: Interoperability -- standardized API contract.
15  */
16 @Data
17 @Builder
18 @AllArgsConstructor
19 @NoArgsConstructor
20 public class ProductResponseDTO {
21
22     private Long id;
23     private String name;
24     private BigDecimal price;
25     private Integer stock;
26     private ProductStatus status;
27     private Long categoryId;
28     private LocalDateTime createdAt;
29 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/ProductUpdateDTO.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import jakarta.validation.constraints.DecimalMin;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.NotNull;
6 import jakarta.validation.constraints.Size;
7 import lombok.AllArgsConstructor;
8 import lombok.Builder;
```

```
9   import lombok.Data;
10  import lombok.NoArgsConstructor;
11
12  import java.math.BigDecimal;
13
14  /**
15   * DTO for updating a product.
16   * McCall Factor: Correctness -- validated partial updates.
17   */
18  @Data
19  @Builder
20  @AllArgsConstructor
21  @NoArgsConstructor
22  public class ProductUpdatedDTO {
23
24      @NotBlank(message = "Product name is required")
25      @Size(max = 150, message = "Product name must not exceed 150 characters")
26      private String name;
27
28      @NotNull(message = "Price is required")
29      @DecimalMin(value = "0.00", message = "Price must be non-negative")
30      private BigDecimal price;
31
32      private Long categoryId;
33  }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementCreateDTO.java

```
Código fuente

1  package com.dongato.inventory.interfaces.rest.dto;
2
3  import com.dongato.inventory.domain.model.MovementReason;
4  import com.dongato.inventory.domain.model.MovementType;
5  import jakarta.validation.constraints.Min;
6  import jakarta.validation.constraints.NotNull;
7  import lombok.AllArgsConstructor;
8  import lombok.Builder;
9  import lombok.Data;
10 import lombok.NoArgsConstructor;
11
12 /**
13  * DTO for creating a stock movement.
14  * McCall Factor: Integrity -- validates audit data at boundary.
15  */
16 @Data
17 @Builder
18 @AllArgsConstructor
19 @NoArgsConstructor
20 public class StockMovementCreateDTO {
21
22     @NotNull(message = "Product ID is required")
23     private Long productId;
24
25     @NotNull(message = "Movement type is required (ENTRADA or SALIDA)")
26     private MovementType type;
27
28     @NotNull(message = "Quantity is required")
29     @Min(value = 1, message = "Quantity must be at least 1")
30     private Integer quantity;
31
32     @NotNull(message = "Movement reason is required")
33     private MovementReason reason;
34 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/dto/StockMovementResponseDTO.java

Código fuente

```
1 package com.dongato.inventory.interfaces.rest.dto;
2
3 import com.dongato.inventory.domain.model.MovementReason;
4 import com.dongato.inventory.domain.model.MovementType;
5 import lombok.AllArgsConstructor;
6 import lombok.Builder;
7 import lombok.Data;
8 import lombok.NoArgsConstructor;
9
10 import java.time.LocalDateTime;
11
12 /**
13  * DTO for stock movement responses.
14  * McCall Factor: Interoperability -- consistent API response format.
15  */
16 @Data
17 @Builder
18 @AllArgsConstructor
19 @NoArgsConstructor
20 public class StockMovementResponseDTO {
21
22     private Long id;
23     private Long productId;
24     private MovementType type;
25     private Integer quantity;
26     private MovementReason reason;
27     private LocalDateTime createdAt;
28 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/exception/GlobalExceptionHandler.java

Código fuente

```
1 package com.dongato.inventory.interfaces.rest.exception;
2
3 import com.dongato.inventory.domain.exception.BusinessRuleException;
4 import com.dongato.inventory.domain.exception.ResourceNotFoundException;
5 import com.dongato.inventory.interfaces.rest.dto.ApiErrorDTO;
6 import jakarta.servlet.http.HttpServletRequest;
7 import lombok.extern.slf4j.Slf4j;
8 import org.springframework.http.HttpStatus;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.security.authentication.BadCredentialsException;
11 import org.springframework.validation.FieldError;
12 import org.springframework.web.bind.MethodArgumentNotValidException;
13 import org.springframework.web.bind.annotation.ExceptionHandler;
14 import org.springframework.web.bind.annotation.RestControllerAdvice;
15
16 import java.time.LocalDateTime;
17 import java.util.List;
18 import java.util.stream.Collectors;
19
20 /**
21  * Global exception handler for the REST API.
22  * <p>
23  * McCall Factors:
24  * - Usability: consistent, helpful error responses
25  * - Integrity: no stack traces leaked to clients
26  */
27 @RestControllerAdvice
28 @Slf4j
29 public class GlobalExceptionHandler {
30
31     @ExceptionHandler(ResourceNotFoundException.class)
32     public ResponseEntity<ApiErrorDTO> handleNotFound(
33         ResourceNotFoundException ex, HttpServletRequest request) {
```

```

34     log warn("Resource not found: {}", ex.getMessage());
35     return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage(), request);
36 }
37
38 @ExceptionHandler BusinessException.class
39 public ResponseEntity<ApiErrorDTO> handleBusinessRule(
40     BusinessException ex, HttpServletRequest request) {
41     log warn("Business rule violation: {}", ex.getMessage());
42     return buildResponse(HttpStatus.CONFLICT, ex.getMessage(), request);
43 }
44
45 @ExceptionHandler IllegalArgumentException.class
46 public ResponseEntity<ApiErrorDTO> handleIllegalArgument(
47     IllegalArgumentException ex, HttpServletRequest request) {
48     log warn("Invalid argument: {}", ex.getMessage());
49     return buildResponse(HttpStatus.BAD_REQUEST, ex.getMessage(), request);
50 }
51
52 @ExceptionHandler MethodArgumentNotValidException.class
53 public ResponseEntity<ApiErrorDTO> handleValidation(
54     MethodArgumentNotValidException ex, HttpServletRequest request) {
55     List<ApiErrorDTO.FieldErrorDTO> fieldErrors = ex.getBindingResult().
56         getFieldErrors().stream()
57         .map(this::mapFieldError)
58         .collect(Collectors.toList());
59
60     ApiErrorDTO error = ApiErrorDTO.builder()
61         .status(HttpStatus.BAD_REQUEST.value())
62         .error("Validation Failed")
63         .message("One or more fields have validation errors")
64         .path(request.getRequestURI())
65         .timestamp(LocalDateTime.now())
66         .fieldErrors(fieldErrors)
67         .build();
68
69     return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
70 }
71
72 @ExceptionHandler BadCredentialsException.class
73 public ResponseEntity<ApiErrorDTO> handleBadCredentials(
74     BadCredentialsException ex, HttpServletRequest request) {
75     log warn("Authentication failed: {}", ex.getMessage());
76     return buildResponse(HttpStatus.UNAUTHORIZED, "Invalid username or password", request);
77 }
78
79 @ExceptionHandler Exception.class
80 public ResponseEntity<ApiErrorDTO> handleGeneric(
81     Exception ex, HttpServletRequest request) {
82     log error("Unexpected error: {}", ex.getMessage(), ex);
83     // Never expose internal details to client (Integrity)
84     return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR,
85         "An unexpected error occurred", request);
86 }
87
88 private ResponseEntity<ApiErrorDTO> buildResponse(
89     HttpStatus status, String message, HttpServletRequest request) {
90     ApiErrorDTO error = ApiErrorDTO.builder()
91         .status(status.value())
92         .error(status.getReasonPhrase())
93         .message(message)
94         .path(request.getRequestURI())
95         .timestamp(LocalDateTime.now())
96         .build();
97     return ResponseEntity.status(status).body(error);
98 }
99
100 private ApiErrorDTO.FieldErrorDTO mapFieldError(FieldError fieldError) {
101     return ApiErrorDTO.FieldErrorDTO.builder()
102         .field(fieldError.getField())
103         .message(fieldError.getDefaultMessage())
104         .build();
105 }
106 }

```

backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/CategoryRestMapper.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.mapper;
2
3 import com.dongato.inventory.domain.model.Category;
4 import com.dongato.inventory.interfaces.rest.dto.CategoryCreateDTO;
5 import com.dongato.inventory.interfaces.rest.dto.CategoryResponseDTO;
6
7 /**
8  * Maps between Category domain model and REST DTOs.
9  * McCall Factor: Maintainability -- isolates API contract from domain.
10 */
11 public final class CategoryRestMapper {
12
13     private CategoryRestMapper() {
14         // Utility class
15     }
16
17     public static Category toDomain(CategoryCreateDTO dto) {
18         return Category.builder()
19             .name(dto.getName())
20             .description(dto.getDescription())
21             .build();
22     }
23
24     public static CategoryResponseDTO toResponseDto(Category domain) {
25         return CategoryResponseDTO.builder()
26             .id(domain.getId())
27             .name(domain.getName())
28             .description(domain.getDescription())
29             .createdAt(domain.getCreatedAt())
30             .build();
31     }
32 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/ProductRestMapper.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.mapper;
2
3 import com.dongato.inventory.domain.model.Product;
4 import com.dongato.inventory.interfaces.rest.dto.ProductCreateDTO;
5 import com.dongato.inventory.interfaces.rest.dto.ProductResponseDTO;
6 import com.dongato.inventory.interfaces.rest.dto.ProductUpdateDTO;
7
8 /**
9  * Maps between Product domain model and REST DTOs.
10  * McCall Factor: Maintainability -- separates API layer from domain.
11 */
12 public final class ProductRestMapper {
13
14     private ProductRestMapper() {
15         // Utility class
16     }
17
18     public static Product toDomain(ProductCreateDTO dto) {
19         return Product.builder()
20             .name(dto.getName())
21             .price(dto.getPrice())
22             .stock(dto.getStock() != null ? dto.getStock() : 0)
23             .categoryId(dto.getCategoryId())
24             .build();
25     }
26
27     public static Product toDomain(ProductUpdateDTO dto) {
28         return Product.builder()
29             .name(dto.getName())
```



```
30         price(dto.getPrice())
31         categoryId dto.getCategoryId()
32         build();
33     }
34
35     public static ProductResponseDTO toResponseDto(Product domain) {
36         return ProductResponseDTO.builder()
37             .id(domain.getId())
38             .name(domain.getName())
39             .price(domain.getPrice())
40             .stock(domain.getStock())
41             .status(domain.getStatus())
42             .categoryId(domain.getCategoryId())
43             .createdAt(domain.getCreatedAt())
44             .build();
45     }
46 }
```

backend/src/main/java/com/dongato/inventory/interfaces/rest/mapper/StockMovementRestMapper.java

```
Código fuente

1 package com.dongato.inventory.interfaces.rest.mapper;
2
3 import com.dongato.inventory.domain.model.StockMovement;
4 import com.dongato.inventory.interfaces.rest.dto.StockMovementCreatedDTO;
5 import com.dongato.inventory.interfaces.rest.dto.StockMovementResponseDTO;
6
7 /**
8  * Maps between StockMovement domain model and REST DTOs.
9  * McCall Factor: Maintainability -- isolates API contract from domain.
10 */
11 public final class StockMovementRestMapper {
12
13     private StockMovementRestMapper() {
14         // Utility class
15     }
16
17     public static StockMovement toDomain(StockMovementCreatedDTO dto) {
18         return StockMovement.builder()
19             .productId(dto.getProductId())
20             .type(dto.getType())
21             .quantity(dto.getQuantity())
22             .reason(dto.getReason())
23             .build();
24     }
25
26     public static StockMovementResponseDTO toResponseDto(StockMovement domain) {
27         return StockMovementResponseDTO.builder()
28             .id(domain.getId())
29             .productId(domain.getProductId())
30             .type(domain.getType())
31             .quantity(domain.getQuantity())
32             .reason(domain.getReason())
33             .createdAt(domain.getCreatedAt())
34             .build();
35     }
36 }
```

A.9 SQA

backend/src/main/java/com/dongato/inventory/sqa/QualityScoringEngine.java

Código fuente

```
1 package com.dongato.inventory.sqa;
2
3 /**
4  * Quality Scoring Engine based on the McCall Quality Model.
5  * <p>
6  * Calculates a composite quality score from metrics collected during CI/CD.
7  * Each factor is weighted according to project priorities.
8  *
9  * McCall Factors measured:
10  * - Testability:      Code coverage percentage (JaCoCo)
11  * - Correctness:      Bug count (SpotBugs)
12  * - Maintainability:  Code smell count (SonarQube)
13  * - Integrity:        Vulnerability count (OWASP Dependency Check)
14  */
15 public class QualityScoringEngine {
16
17     // Weights assigned based on project importance
18     private static final double WEIGHT_CORRECTNESS      = 0.30;
19     private static final double WEIGHT_TESTABILITY      = 0.30;
20     private static final double WEIGHT_MAINTAINABILITY   = 0.20;
21     private static final double WEIGHT_INTEGRITY        = 0.20;
22
23     private QualityScoringEngine() {
24         // Utility class -- prevent instantiation
25     }
26
27     /**
28      * Calculates the overall quality score (0-100).
29      *
30      * @param coveragePercent code coverage percentage from JaCoCo
31      * @param bugs             bug count from SpotBugs
32      * @param smells           code smell count from SonarQube
33      * @param vulns            vulnerability count from OWASP Dependency Check
34      * @return composite quality score (0-100)
35      */
36     public static double calculateScore(double coveragePercent, int bugs, int smells, int vulns) {
37         if (coveragePercent < 0 || coveragePercent > 100) {
38             throw new IllegalArgumentException("Coverage must be between 0 and 100");
39         }
40         if (bugs < 0 || smells < 0 || vulns < 0) {
41             throw new IllegalArgumentException("Metric counts cannot be negative");
42         }
43
44         double testabilityScore      = Math.min(coveragePercent, 100.0);
45         double correctnessScore      = Math.max(100.0 - (bugs * 10.0), 0.0);
46         double maintainabilityScore  = Math.max(100.0 - (smells * 2.0), 0.0);
47         double integrityScore        = vulns == 0 ? 100.0 : 0.0; // Zero tolerance
48
49         return (testabilityScore * WEIGHT_TESTABILITY) +
50             (correctnessScore * WEIGHT_CORRECTNESS) +
51             (maintainabilityScore * WEIGHT_MAINTAINABILITY) +
52             (integrityScore * WEIGHT_INTEGRITY);
53     }
54
55     /**
56      * Returns a human-readable quality grade.
57      */
58     public static String getGrade(double score) {
59         if (score >= 90) return "A (Excellent)";
60         if (score >= 80) return "B (Good)";
61         if (score >= 70) return "C (Acceptable)";
62         if (score >= 60) return "D (Needs Improvement)";
63         return "F (Critical)";
64     }
65 }
```

A.10 Pruebas

backend/src/test/java/com/dongato/inventory/DonGatoInventoryApplicationTests.java

```
Código fuente

1 package com.dongato.inventory;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.springframework.test.context.ActiveProfiles;
6 import org.springframework.test.context.TestPropertySource;
7
8 @SpringBootTest
9 @ActiveProfiles("test")
10 @TestPropertySource(properties = {
11     "spring.docker.compose.enabled=false"
12 })
13 class DonGatoInventoryApplicationTests {
14
15     @Test
16     void contextLoads() {
17         // Verifies that the Spring application context loads successfully
18     }
19 }
```

backend/src/test/java/com/dongato/inventory/application/usecases/CategoryUseCaseTest.java

```
Código fuente

1 package com.dongato.inventory.application.usecases;
2
3 import com.dongato.inventory.application.ports.out.CategoryRepositoryPort;
4 import com.dongato.inventory.domain.exception.BusinessRuleException;
5 import com.dongato.inventory.domain.exception.ResourceNotFoundException;
6 import com.dongato.inventory.domain.model.Category;
7 import org.junit.jupiter.api.DisplayName;
8 import org.junit.jupiter.api.Test;
9 import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.InjectMocks;
11 import org.mockito.Mock;
12 import org.mockito.junit.jupiter.MockitoExtension;
13
14 import java.util.List;
15 import java.util.Optional;
16
17 import static org.junit.jupiter.api.Assertions.*;
18 import static org.mockito.ArgumentMatchers.any;
19 import static org.mockito.Mockito.*;
20
21 /**
22  * Unit tests for CategoryUseCase.
23  * McCall Factor: Testability -- isolated business logic tests.
24  */
25 @ExtendWith(MockitoExtension.class)
26 class CategoryUseCaseTest {
27
28     @Mock
29     private CategoryRepositoryPort categoryRepository;
30
31     @InjectMocks
32     private CategoryUseCase categoryUseCase;
33
34     @Test
35     @DisplayName("Should create category successfully")
36     void shouldCreateCategorySuccessfully() {
37         Category input = Category.builder().name("CAFE").description("Cafés").build();
38         Category saved = Category.builder().id(1L).name("CAFE").description("Cafés").build();
```

```

39
40     when(categoryRepository existsByName("CAFE")).thenReturn(false);
41     when(categoryRepository save(any(Category class))) thenReturn(saved);
42
43     Category result = categoryUseCase create(input);
44
45     assertNotNull(result);
46     assertEquals(1L, result.getId());
47     assertEquals("CAFE", result.getName());
48     verify(categoryRepository) save(any(Category class));
49 }
50
51 @Test
52 @DisplayName("Should throw when creating duplicate category")
53 void shouldThrowOnDuplicateCategory() {
54     Category input = Category.builder().name("CAFE").build();
55     when(categoryRepository existsByName("CAFE")).thenReturn(true);
56
57     assertThrows BusinessRuleException.class,
58         () -> categoryUseCase create(input);
59     verify(categoryRepository, never()).save(any());
60 }
61
62 @Test
63 @DisplayName("Should find category by ID")
64 void shouldFindById() {
65     Category expected = Category.builder().id(1L).name("CAFE").build();
66     when(categoryRepository findById(1L)).thenReturn(Optional.of(expected));
67
68     Category result = categoryUseCase findById(1L);
69
70     assertEquals("CAFE", result.getName());
71 }
72
73 @Test
74 @DisplayName("Should throw when category not found")
75 void shouldThrowWhenNotFound() {
76     when(categoryRepository findById(99L)).thenReturn(Optional.empty());
77
78     assertThrows ResourceNotFoundException.class,
79         () -> categoryUseCase findById(99L);
80 }
81
82 @Test
83 @DisplayName("Should return all categories")
84 void shouldFindAll() {
85     when(categoryRepository findAll()).thenReturn(
86         List.of(
87             Category.builder().id(1L).name("CAFE").build(),
88             Category.builder().id(2L).name("BEBIDA").build()
89         ));
90
91     List<Category> result = categoryUseCase findAll();
92     assertEquals(2, result.size());
93 }
94
95 @Test
96 @DisplayName("Should delete category successfully")
97 void shouldDeleteSuccessfully() {
98     when(categoryRepository existsById(1L)).thenReturn(true);
99
100     assertDoesNotThrow(() -> categoryUseCase delete(1L));
101     verify(categoryRepository) deleteById(1L);
102 }
103
104 @Test
105 @DisplayName("Should throw when deleting non-existent category")
106 void shouldThrowWhenDeletingNonExistent() {
107     when(categoryRepository existsById(99L)).thenReturn(false);
108
109     assertThrows ResourceNotFoundException.class,
110         () -> categoryUseCase delete(99L);
111 }
112 }

```

backend/src/test/java/com/dongato/inventory/application/usecases/StockMovementUseCaseTest.java

Código fuente

```
1 package com.dongato.inventory.application.usecases;
2
3 import com.dongato.inventory.application.ports.out.ProductRepositoryPort;
4 import com.dongato.inventory.application.ports.out.StockMovementRepositoryPort;
5 import com.dongato.inventory.domain.exception.InsufficientStockException;
6 import com.dongato.inventory.domain.exception.ResourceNotFoundException;
7 import com.dongato.inventory.domain.model.*;
8 import org.junit.jupiter.api.DisplayName;
9 import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.InjectMocks;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.math.BigDecimal;
16 import java.util.Optional;
17
18 import static org.junit.jupiter.api.Assertions.*;
19 import static org.mockito.ArgumentMatchers.any;
20 import static org.mockito.Mockito.*;
21
22 /**
23  * Unit tests for StockMovementUseCase.
24  * McCall Factor: Testability -- validates stock movement logic in isolation.
25  */
26 @ExtendWith(MockitoExtension.class)
27 class StockMovementUseCaseTest {
28
29     @Mock
30     private StockMovementRepositoryPort movementRepository;
31
32     @Mock
33     private ProductRepositoryPort productRepository;
34
35     @InjectMocks
36     private StockMovementUseCase stockMovementUseCase;
37
38     @Test
39     @DisplayName("Should register ENTRADA and increase product stock")
40     void shouldRegisterEntradaSuccessfully() {
41         Product product = Product.builder()
42             .id(1L).name("Americano").price(new BigDecimal("2.50"))
43             .stock(10).status(ProductStatus.AVAILABLE).build();
44         StockMovement movement = StockMovement.builder()
45             .productId(1L).type(MovementType.ENTRADA)
46             .quantity(5).reason(MovementReason.REPOSICION).build();
47         StockMovement saved = StockMovement.builder()
48             .id(1L).productId(1L).type(MovementType.ENTRADA)
49             .quantity(5).reason(MovementReason.REPOSICION).build();
50
51         when(productRepository.findById(1L)).thenReturn(Optional.of(product));
52         when(productRepository.save(any(Product.class))).thenReturn(product);
53         when(movementRepository.save(any(StockMovement.class))).thenReturn(saved);
54
55         StockMovement result = stockMovementUseCase.registerMovement(movement);
56
57         assertNotNull(result);
58         assertEquals(15, product.getStock());
59         assertEquals(ProductStatus.AVAILABLE, product.getStatus());
60         verify(productRepository).save(product);
61         verify(movementRepository).save(any(StockMovement.class));
62     }
63
64     @Test
65     @DisplayName("Should register SALIDA and decrease product stock")
66     void shouldRegisterSalidaSuccessfully() {
67         Product product = Product.builder()
68             .id(1L).name("Latte").price(new BigDecimal("3.75"))
69             .stock(10).status(ProductStatus.AVAILABLE).build();
70         StockMovement movement = StockMovement.builder()
71             .productId(1L).type(MovementType.SALIDA)
```

```

72         quantity(3).reason MovementReason.VENTA).build();
73     StockMovement saved = StockMovement.builder()
74         .id(1L) .productId(1L) .type MovementType.SALIDA
75         .quantity(3).reason MovementReason.VENTA).build();
76
77     when(productRepository.findById(1L)).thenReturn(Optional.of(product));
78     when(productRepository.save(any(Product.class))).thenReturn(product);
79     when(movementRepository.save(any(StockMovement.class))).thenReturn(saved);
80
81     StockMovement result = stockMovementUseCase.registerMovement(movement);
82
83     assertNotNull(result);
84     assertEquals(7, product.getStock());
85     verify(productRepository).save(product);
86 }
87
88 @Test
89 @DisplayName("Should throw InsufficientStockException on SALIDA with insufficient stock")
90 void shouldThrowOnInsufficientStock() {
91     Product product = Product.builder()
92         .id(1L) .name("Espresso") .price(new BigDecimal("2.00"))
93         .stock(2).status ProductStatus.AVAILABLE).build();
94     StockMovement movement = StockMovement.builder()
95         .productId(1L) .type MovementType.SALIDA
96         .quantity(5).reason MovementReason.VENTA).build();
97
98     when(productRepository.findById(1L)).thenReturn(Optional.of(product));
99
100    assertThrows(InsufficientStockException.class,
101        () -> stockMovementUseCase.registerMovement(movement));
102    verify(productRepository, never()).save(any());
103    verify(movementRepository, never()).save(any());
104 }
105
106 @Test
107 @DisplayName("Should throw when product not found")
108 void shouldThrowWhenProductNotFound() {
109     StockMovement movement = StockMovement.builder()
110         .productId(99L) .type MovementType.ENTRADA
111         .quantity(5).reason MovementReason.REPOSICION).build();
112
113     when(productRepository.findById(99L)).thenReturn(Optional.empty());
114
115     assertThrows(ResourceNotFoundException.class,
116         () -> stockMovementUseCase.registerMovement(movement));
117 }
118
119 @Test
120 @DisplayName("Should set status to OUT_OF_STOCK when stock reaches zero")
121 void shouldSetOutOfStockWhenZero() {
122     Product product = Product.builder()
123         .id(1L) .name("Mocha") .price(new BigDecimal("4.00"))
124         .stock(5).status ProductStatus.AVAILABLE).build();
125     StockMovement movement = StockMovement.builder()
126         .productId(1L) .type MovementType.SALIDA
127         .quantity(5).reason MovementReason.VENTA).build();
128     StockMovement saved = StockMovement.builder()
129         .id(1L) .productId(1L) .type MovementType.SALIDA
130         .quantity(5).reason MovementReason.VENTA).build();
131
132     when(productRepository.findById(1L)).thenReturn(Optional.of(product));
133     when(productRepository.save(any(Product.class))).thenReturn(product);
134     when(movementRepository.save(any(StockMovement.class))).thenReturn(saved);
135
136     stockMovementUseCase.registerMovement(movement);
137
138     assertEquals(0, product.getStock());
139     assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
140 }
141 }

```

backend/src/test/java/com/dongato/inventory/domain/model/CategoryTest.java

```
Código fuente

1 package com.dongato.inventory.domain.model;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 /**
9  * Unit tests for Category domain model.
10  * McCall Factor: Testability -- validates domain invariants.
11  */
12 class CategoryTest {
13
14     @Test
15     @DisplayName("Should validate successfully with valid data")
16     void shouldValidateSuccessfully() {
17         Category category = Category.builder()
18             .name("CAFE")
19             .description("Bebidas de café")
20             .build();
21
22         assertDoesNotThrow(category::validate);
23     }
24
25     @Test
26     @DisplayName("Should throw on null name")
27     void shouldThrowOnNullName() {
28         Category category = Category.builder()
29             .name(null)
30             .build();
31
32         assertThrows(IllegalArgumentException.class, category::validate);
33     }
34
35     @Test
36     @DisplayName("Should throw on blank name")
37     void shouldThrowOnBlankName() {
38         Category category = Category.builder()
39             .name(" ")
40             .build();
41
42         assertThrows(IllegalArgumentException.class, category::validate);
43     }
44 }
```

backend/src/test/java/com/dongato/inventory/domain/model/ProductTest.java

```
Código fuente

1 package com.dongato.inventory.domain.model;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Nested;
5 import org.junit.jupiter.api.Test;
6
7 import java.math.BigDecimal;
8
9 import static org.junit.jupiter.api.Assertions.*;
10
11 /**
12  * Unit tests for Product domain model.
13  * McCall Factor: Testability -- validates business rules in isolation.
14  */
15 class ProductTest {
16
17     @Nested
```

```
18 @DisplayName("Stock Management")
19 class StockManagement {
20
21     @Test
22     @DisplayName("Should add stock and recalculate status to AVAILABLE")
23     void shouldAddStockSuccessfully() {
24         Product product = Product.builder()
25             .name("Americano")
26             .price(new BigDecimal("2.50"))
27             .stock(0)
28             .status(ProductStatus.OUT_OF_STOCK)
29             .build();
30
31         product.addStock(10);
32
33         assertEquals(10, product.getStock());
34         assertEquals(ProductStatus.AVAILABLE, product.getStatus());
35     }
36
37     @Test
38     @DisplayName("Should remove stock and recalculate status")
39     void shouldRemoveStockSuccessfully() {
40         Product product = Product.builder()
41             .name("Cappuccino")
42             .price(new BigDecimal("3.50"))
43             .stock(5)
44             .status(ProductStatus.AVAILABLE)
45             .build();
46
47         product.removeStock(5);
48
49         assertEquals(0, product.getStock());
50         assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
51     }
52
53     @Test
54     @DisplayName("Should throw exception for insufficient stock")
55     void shouldThrowOnInsufficientStock() {
56         Product product = Product.builder()
57             .name("Latte")
58             .price(new BigDecimal("3.75"))
59             .stock(3)
60             .build();
61
62         IllegalStateException ex = assertThrows(
63             IllegalStateException.class,
64             () -> product.removeStock(5));
65
66         assertTrue(ex.getMessage().contains("Insufficient stock"));
67     }
68
69     @Test
70     @DisplayName("Should throw exception for negative entry quantity")
71     void shouldThrowOnNegativeEntryQuantity() {
72         Product product = Product.builder()
73             .name("Espresso")
74             .price(new BigDecimal("2.00"))
75             .stock(10)
76             .build();
77
78         assertThrows(IllegalArgumentException.class,
79             () -> product.addStock(-5));
80     }
81
82     @Test
83     @DisplayName("Should throw exception for zero exit quantity")
84     void shouldThrowOnZeroExitQuantity() {
85         Product product = Product.builder()
86             .name("Mocha")
87             .price(new BigDecimal("4.00"))
88             .stock(10)
89             .build();
90
91         assertThrows(IllegalArgumentException.class,
92             () -> product.removeStock(0));
93     }
94 }
```



```

93     }
94 }
95
96 @Nested
97 @DisplayName("Status Recalculation")
98 class StatusRecalculation {
99
100     @Test
101     @DisplayName("Should set AVAILABLE when stock > 0")
102     void shouldBeAvailableWithPositiveStock() {
103         Product product = Product.builder().stock(1).build();
104         product.recalculateStatus();
105         assertEquals(ProductStatus.AVAILABLE, product.getStatus());
106     }
107
108     @Test
109     @DisplayName("Should set OUT_OF_STOCK when stock = 0")
110     void shouldBeOutOfStockWhenZero() {
111         Product product = Product.builder().stock(0).build();
112         product.recalculateStatus();
113         assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
114     }
115
116     @Test
117     @DisplayName("Should set OUT_OF_STOCK when stock is null")
118     void shouldBeOutOfStockWhenNull() {
119         Product product = Product.builder().stock(null).build();
120         product.recalculateStatus();
121         assertEquals(ProductStatus.OUT_OF_STOCK, product.getStatus());
122     }
123 }
124
125 @Nested
126 @DisplayName("Validation")
127 class Validation {
128
129     @Test
130     @DisplayName("Should throw on blank name")
131     void shouldThrowOnBlankName() {
132         Product product = Product.builder()
133             .name("")
134             .price(new BigDecimal("2.50"))
135             .stock(0)
136             .build();
137
138         assertThrows(IllegalArgumentException.class, product::validate);
139     }
140
141     @Test
142     @DisplayName("Should throw on negative price")
143     void shouldThrowOnNegativePrice() {
144         Product product = Product.builder()
145             .name("Test")
146             .price(new BigDecimal("-1.00"))
147             .stock(0)
148             .build();
149
150         assertThrows(IllegalArgumentException.class, product::validate);
151     }
152
153     @Test
154     @DisplayName("Should throw on negative stock")
155     void shouldThrowOnNegativeStock() {
156         Product product = Product.builder()
157             .name("Test")
158             .price(new BigDecimal("2.50"))
159             .stock(-5)
160             .build();
161
162         assertThrows(IllegalArgumentException.class, product::validate);
163     }
164
165     @Test
166     @DisplayName("Should pass validation for valid product")
167     void shouldPassValidation() {

```

```
168         Product product = Product builder()
169             name("Americano")
170             price(new BigDecimal("2.50"))
171             stock(10)
172             build();
173
174         assertDoesNotThrow(product::validate);
175     }
176 }
177 }
```

backend/src/test/java/com/dongato/inventory/domain/model/StockMovementTest.java

```
Código fuente

1 package com.dongato.inventory.domain.model;
2
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 /**
9  * Unit tests for StockMovement domain model.
10  * McCall Factor: Testability -- validates audit data integrity.
11  */
12 class StockMovementTest {
13
14     @Test
15     @DisplayName("Should validate successfully with valid data")
16     void shouldValidateSuccessfully() {
17         StockMovement movement = StockMovement.builder()
18             .productId(1L)
19             .type(MovementType.ENTRADA)
20             .quantity(10)
21             .reason(MovementReason.REPOSICION)
22             .build();
23
24         assertDoesNotThrow(movement::validate);
25     }
26
27     @Test
28     @DisplayName("Should throw on null product ID")
29     void shouldThrowOnNullProductId() {
30         StockMovement movement = StockMovement.builder()
31             .productId(null)
32             .type(MovementType.ENTRADA)
33             .quantity(10)
34             .reason(MovementReason.REPOSICION)
35             .build();
36
37         assertThrows(IllegalArgumentException.class, movement::validate);
38     }
39
40     @Test
41     @DisplayName("Should throw on null type")
42     void shouldThrowOnNullType() {
43         StockMovement movement = StockMovement.builder()
44             .productId(1L)
45             .type(null)
46             .quantity(10)
47             .reason(MovementReason.VENTA)
48             .build();
49
50         assertThrows(IllegalArgumentException.class, movement::validate);
51     }
52
53     @Test
54     @DisplayName("Should throw on zero quantity")
55     void shouldThrowOnZeroQuantity() {
56         StockMovement movement = StockMovement.builder()
```

```

57         productId(1L)
58         type MovementType SALIDA
59         quantity(0)
60         reason(MovementReason VENTA)
61         build();
62
63         assertThrows(IllegalArgumentException.class, movement::validate);
64     }
65
66     @Test
67     @DisplayName("Should throw on null reason")
68     void shouldThrowOnNullReason() {
69         StockMovement movement = StockMovement.builder()
70             productId(1L)
71             type MovementType ENTRADA
72             quantity(10)
73             reason(null)
74             build();
75
76         assertThrows(IllegalArgumentException.class, movement::validate);
77     }
78 }

```

backend/src/test/java/com/dongato/inventory/sqa/QualityScoringEngineTest.java

Código fuente

```

1  package com.dongato.inventory.sqa;
2
3  import org.junit.jupiter.api.DisplayName;
4  import org.junit.jupiter.api.Test;
5
6  import static org.junit.jupiter.api.Assertions.*;
7
8  /**
9   * Unit tests for QualityScoringEngine.
10   * McCall Factor: Testability -- validates quality scoring calculations.
11   */
12  class QualityScoringEngineTest {
13
14      @Test
15      @DisplayName("Should return perfect score with ideal metrics")
16      void shouldReturnPerfectScore() {
17          double score = QualityScoringEngine.calculateScore(100.0, 0, 0, 0);
18          assertEquals(100.0, score, 0.01);
19          assertEquals("A (Excellent)", QualityScoringEngine.getGrade(score));
20      }
21
22      @Test
23      @DisplayName("Should return zero integrity score with vulnerabilities")
24      void shouldPenalizeVulnerabilities() {
25          double withVulns = QualityScoringEngine.calculateScore(100.0, 0, 0, 1);
26          double withoutVulns = QualityScoringEngine.calculateScore(100.0, 0, 0, 0);
27
28          assertTrue(withVulns < withoutVulns);
29          assertEquals(80.0, withVulns, 0.01); // 100*0.3 + 100*0.3 + 100*0.2 + 0*0.2
30      }
31
32      @Test
33      @DisplayName("Should degrade correctness score with bugs")
34      void shouldDegradeWithBugs() {
35          double score = QualityScoringEngine.calculateScore(100.0, 5, 0, 0);
36          // Correctness: max(100 - 50, 0) = 50
37          // 100*0.3 + 50*0.3 + 100*0.2 + 100*0.2 = 30+15+20+20 = 85
38          assertEquals(85.0, score, 0.01);
39      }
40
41      @Test
42      @DisplayName("Should degrade maintainability with code smells")
43      void shouldDegradeWithSmells() {
44          double score = QualityScoringEngine.calculateScore(100.0, 0, 25, 0);

```

```
45         // Maintainability:  $\max(100 - 50, 0) = 50$ 
46         //  $100 \cdot 0.3 + 100 \cdot 0.3 + 50 \cdot 0.2 + 100 \cdot 0.2 = 30 + 30 + 10 + 20 = 90$ 
47         assertEquals(90.0, score, 0.01);
48     }
49
50     @Test
51     @DisplayName("Should throw on invalid coverage")
52     void shouldThrowOnInvalidCoverage() {
53         assertThrows(IllegalArgumentException.class,
54             () -> QualityScoringEngine.calculateScore(-1.0, 0, 0, 0));
55         assertThrows(IllegalArgumentException.class,
56             () -> QualityScoringEngine.calculateScore(101.0, 0, 0, 0));
57     }
58
59     @Test
60     @DisplayName("Should throw on negative metrics")
61     void shouldThrowOnNegativeMetrics() {
62         assertThrows(IllegalArgumentException.class,
63             () -> QualityScoringEngine.calculateScore(80.0, -1, 0, 0));
64     }
65
66     @Test
67     @DisplayName("Should grade correctly at boundaries")
68     void shouldGradeCorrectly() {
69         assertEquals("A (Excellent)", QualityScoringEngine.getGrade(90));
70         assertEquals("B (Good)", QualityScoringEngine.getGrade(85));
71         assertEquals("C (Acceptable)", QualityScoringEngine.getGrade(75));
72         assertEquals("D (Needs Improvement)", QualityScoringEngine.getGrade(65));
73         assertEquals("F (Critical)", QualityScoringEngine.getGrade(50));
74     }
75 }
```

Apéndice B

Snapshots de Archivos Eliminados

B.1 Snapshot de aplicación Spring anterior

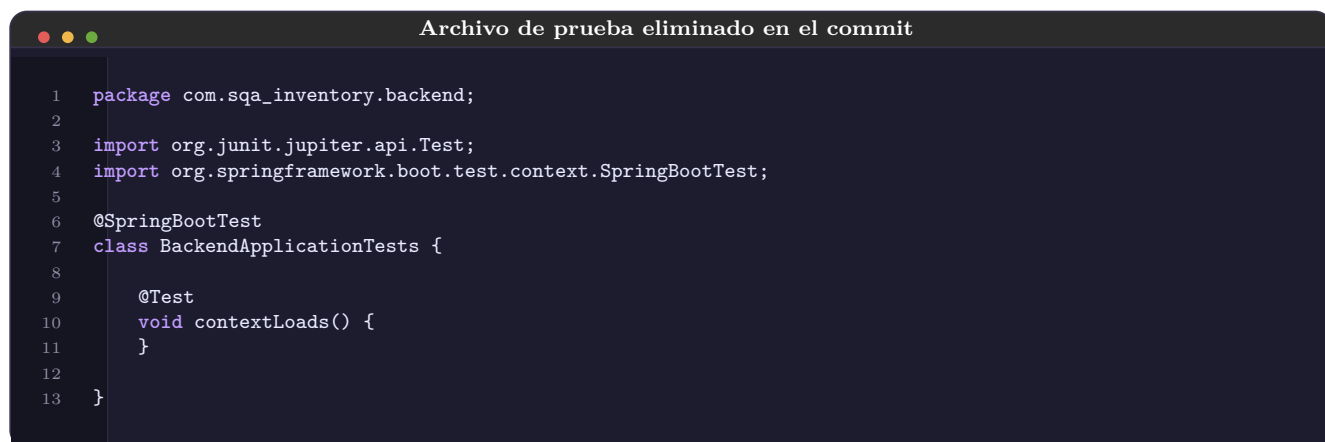
backend/src/main/java/com/sqa_inventory/backend/BackendApplication.java



```
1 package com.sqa_inventory.backend;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class BackendApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(BackendApplication.class, args);
11     }
12
13 }
```

B.2 Snapshot de prueba anterior

backend/src/test/java/com/sqa_inventory/backend/BackendApplicationTests.java



```
1 package com.sqa_inventory.backend;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class BackendApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
```